

Formal Verification Report: Parallel Protocol

- Date: June 10, 2026
 - Audit Repo: https://github.com/Cyfrin/audit-2026-02-parallel_3.1
 - Client Repos: <https://github.com/parallel-protocol/parallel-parallelizer>, <https://github.com/parallel-protocol/parallel-tokens>
 - Audit Commit: 2781b6799d7e7cb9946cc681f6ae173cf9fbcd50
 - Mitigation Commit: 7c24bc40d50bd86384a71ca621ffc5a1838d0368
 - Author: [AlexZoid](#) (Cyfrin private formal verification engagement)
 - Certora Prover version: 8.13.0
-

Table of Contents

1. [About Parallel Protocol](#)
 2. [Formal Verification Methodology](#)
 - [Types of Properties](#)
 - [Verification Process](#)
 - [Project Structure](#)
 - [Assumptions](#)
 3. [Verification Properties](#)
 - [Valid State](#)
 - [Variable Transitions](#)
 - [State Transitions](#)
 - [High-Level](#)
 - [Unit Tests](#)
 - [ERC-4626 Conformance](#)
 4. [Real Issues Found](#)
 5. [Mutation Testing](#)
 - [What is Mutation Testing](#)
 - [VS-P-15: normalizedStablesSumConservation](#)
 6. [Setup and Execution](#)
 - [Common Setup \(Steps 1–4\)](#)
 - [Remote Execution](#)
 - [Local Execution](#)
 - [Required source code modifications](#)
 - [Running Verification](#)
 7. [Resources](#)
-

About Parallel Protocol

Parallel is a decentralized stablecoin protocol (fork of Angle Protocol's Transmuter) that lets users mint over-collateralized stablecoins (TokenP) by depositing correlated collateral assets. The core engine ("Parallelizer") uses an EIP-2535 Diamond proxy supporting 1:1 minting/burning with exposure-based piecewise-linear fees, proportional basket redemption, and surplus distribution.

Three formal verification targets were in scope:

1. **Parallelizer** — The Diamond-proxy stablecoin engine with four facets verified independently:

- **Swapper** — Mint TokenP by depositing collateral; burn TokenP to withdraw collateral
 - **Redeemer** — Proportional basket redemption and normalizer updates
 - **Surplus** — Convert collateral surplus to TokenP and distribute to payees
 - **Setter** — Governance functions: add/revoke collateral, pause, set fees, manage trusted roles
2. **Savings** — ERC-4626 yield vault where users deposit TokenP to earn interest accrued via governance-set rates
3. **FlashParallelToken** — ERC-3156 flash lender that temporarily mints TokenP, collects basis-point fees, and burns the principal

Key operations:

- `swapExactInput` / `swapExactOutput` — mint TokenP by depositing collateral or burn TokenP to withdraw collateral, with exposure-based piecewise-linear fees
- `redeem` / `redeemWithForfeit` — proportional basket redemption of TokenP for underlying collateral assets
- `updateNormalizer` — rebase normalizedStables accounting to reflect external yield or loss events
- `processSurplus` / `release` — convert excess collateral to TokenP and distribute to registered payees
- `deposit` / `mint` / `withdraw` / `redeem` (Savings) — ERC-4626 yield vault where TokenP holders earn governance-set interest rates
- `flashLoan` — ERC-3156 flash mint/burn of TokenP with configurable basis-point fees

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

Properties are categorized following the [official Certora methodology](#). Valid State, Variable Transitions and State Transitions properties are **parametric** — they are automatically verified against every external function in the contract, including functions added after the specification is completed. High-Level properties target specific function sequences.

Valid State — System-wide invariants that MUST always hold true. These properties define the fundamental constraints of the protocol, such as accounting consistency and structural integrity. Once proven, these invariants serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

Variable Transitions — Properties that verify specific storage variables change only under expected conditions. The process captures a variable value before instruction execution, runs the instruction, then asserts the variable changed only as permitted or remained unchanged.

State Transitions — Properties that verify the correctness of transitions between valid states. Building upon the valid state invariants, these properties ensure the protocol's state machine operates correctly and that state changes are both authorized and sequentially valid.

High-Level — Complex multi-step properties verifying business logic integrity. Unlike parametric properties, these target specific function sequences to validate end-to-end protocol behavior.

Unit Tests — Properties that verify basic behavior of individual functions: revert conditions, direct effects on state, and non-effects on unrelated state. Unlike parametric properties, each unit test targets a specific function call.

ERC-4626 Conformance — Properties verifying the Savings vault's compliance with the ERC-4626 tokenized vault standard, including asset/totalAssets correctness, conversion functions, preview accuracy, deposit/mint/withdraw/redeem semantics, and allowance management.

Verification Process

1. **Setup phase:** Define ghost variables, storage hooks, and helper definitions to model contract state in CVL. Establish verification harnesses and configurations for three independent targets (Parallelizer with four facets, Savings, FlashParallelToken). This phase addresses several prover limitations:
 - o Diamond proxy storage ([parallelizer_storage.spec](#)): The Parallelizer uses EIP-2535 Diamond storage with a complex `TransmuterStorage` struct containing nested arrays and mappings. Ghost variables and storage hooks mirror all relevant slots — collateral list, fee curves, normalizedStables, payees, and shares — enabling CVL rules to reason about the full protocol state.
 - o External dependency summaries ([parallelizer.spec](#), [collateral_manager.spec](#)): LibOracle, LibManager, LibWhitelist, and LibDiamond contain complex external interactions (price feeds, yield strategies, access control) that cause solver timeouts. These are replaced with ghost-backed CVL functions or `NONDET` summaries that preserve essential behavioral properties while eliminating implementation complexity.
 - o ERC-20 model ([erc20.spec](#), [token_p.spec](#), [safe_erc20.spec](#)): Real ERC-20 contracts cause timeouts due to implementation complexity and unbounded address space. A lightweight ghost-variable-based model summarizes all external ERC-20 calls (`balanceOf`, `transfer`, `transferFrom`, `approve`, `mint`, `burn`) over a bounded account set, enabling tractable solvency invariants.
 - o Math utilities ([math.spec](#)): PercentageMathLib's `percentMul` uses intermediate multiplication that overflows in the solver. It is replaced with the exact CVL expression `(x * pct + 5000) / 10000`.
 - o OpenZeppelin infrastructure ([parallelizer.spec](#)): Initializable, UUPSUpgradeable, and ReentrancyGuardTransient functions are summarized with `NONDET DELETE` or constrained ghost variables to exclude infrastructure code from

verification while preserving reentrancy guard semantics.

2. **Crafting Properties:** Write invariants and rules in CVL, starting with valid state invariants (which become trusted preconditions for other rules), then variable/state transitions, and finally high-level and unit test properties.

Project Structure

```
certora/
├── configs/
│   ├── FlashParallelToken/
│   │   ├── debug/sanity.conf
│   │   ├── high_level.conf
│   │   ├── state_transitions.conf
│   │   ├── unit_tests.conf
│   │   ├── valid_state.conf
│   │   └── variable_transitions.conf
│   ├── Parallelizer/
│   │   ├── debug/test_layout.conf
│   │   ├── valid_state.sh
│   │   ├── state_transitions.sh
│   │   ├── variable_transitions.sh
│   │   └── Redeemer/
│   │       ├── high_level.conf
│   │       ├── state_transitions.conf
│   │       ├── unit_tests.conf
│   │       ├── valid_state.conf
│   │       ├── valid_state_mutations_normalizedStablesSumConservation.conf
│   │       └── variable_transitions.conf
│   ├── Setter/
│   │   ├── high_level.conf
│   │   ├── state_transitions.conf
│   │   ├── unit_tests.conf
│   │   ├── valid_state.conf
│   │   ├── valid_state_mutations_normalizedStablesSumConservation.conf
│   │   └── variable_transitions.conf
│   ├── Surplus/
│   │   ├── high_level.conf
│   │   ├── state_transitions.conf
│   │   ├── unit_tests.conf
│   │   ├── valid_state.conf
│   │   └── variable_transitions.conf
│   └── Swapper/
│       ├── high_level.conf
│       ├── state_transitions.conf
│       ├── unit_tests.conf
│       ├── valid_state.conf
│       ├── valid_state_mutations_normalizedStablesSumConservation.conf
│       └── variable_transitions.conf
├── Savings/
│   ├── debug/
│   ├── erc4626.conf
│   ├── erc4626_donationProfitBoundedByShareRatio.conf
│   ├── high_level.conf
│   ├── state_transitions.conf
│   ├── unit_tests.conf
│   ├── valid_state.conf
│   └── variable_transitions.conf
├── harnesses/
│   ├── debug/
│   ├── FlashParallelTokenHarness.sol
│   ├── HelperCVL.sol
│   ├── ParallelizerLayout.sol
│   └── RedeemerHarness.sol
```

Prover configuration files
Flash loan token specs
Setup sanity check
Diamond proxy specs
Storage layout self-test
Run a spec type across all 4 facets
Redeemer facet
Setter facet (Governor + Guardian)
Surplus facet
Swapper facet
ERC4626 vault specs
Setup sanity / layout self-tests
ERC4626-51, extended solver portfolio
Verification harnesses
Storage layout self-test harnesses
Flash loan interest accrual wrapper
Assertion wrapper for CVL helpers
Diamond storage layout mapping
Redeemer facet + layout harness

- └─ SavingsHarness.sol # Savings + layout getters harness
- └─ SavingsLayout.sol # ERC4626/ERC20 OZ storage mapping
- └─ SetterHarness.sol # Governor + Guardian setter harness
- └─ SurplusHarness.sol # Surplus + Swapper facet harness
- └─ SwapperHarness.sol # Swapper facet harness
- └─ mutations/ # Mutation testing (VS-P-15 case study)
 - └─ add_mutation.sh # Helper to record a mutant from a working-tree diff
 - └─ normalizedStablesSumConservation/ # 18 mutants: Swapper, LibSetters, Redeemer
- └─ specs/ # CVL specifications
 - └─ flashParallelToken/ # FlashParallelToken properties
 - └─ flashParallelToken_high_level.spec
 - └─ flashParallelToken_state_transitions.spec
 - └─ flashParallelToken_unit_tests.spec
 - └─ flashParallelToken_valid_state.spec
 - └─ flashParallelToken_variable_transitions.spec
 - └─ parallelizer/ # Parallelizer properties (shared + per-facet)
 - └─ parallelizer_state_transitions.spec # Cross-facet state transitions
 - └─ parallelizer_valid_state.spec # Cross-facet valid state invariants
 - └─ parallelizer_variable_transitions.spec # Cross-facet variable transitions
 - └─ redeemer/ # Redeemer facet properties
 - └─ redeemer_high_level.spec
 - └─ redeemer_state_transitions.spec
 - └─ redeemer_unit_tests.spec
 - └─ redeemer_valid_state.spec
 - └─ redeemer_variable_transitions.spec
 - └─ setter/ # Setter facet properties
 - └─ setter_high_level.spec
 - └─ setter_state_transitions.spec
 - └─ setter_unit_tests.spec
 - └─ setter_valid_state.spec
 - └─ setter_variable_transitions.spec
 - └─ surplus/ # Surplus facet properties
 - └─ surplus_high_level.spec
 - └─ surplus_state_transitions.spec
 - └─ surplus_unit_tests.spec
 - └─ surplus_valid_state.spec
 - └─ surplus_variable_transitions.spec
 - └─ swapper/ # Swapper facet properties
 - └─ swapper_high_level.spec
 - └─ swapper_state_transitions.spec
 - └─ swapper_unit_tests.spec
 - └─ swapper_valid_state.spec
 - └─ swapper_variable_transitions.spec
 - └─ savings/ # Savings properties
 - └─ savings_erc4626.spec # ERC4626 compliance rules
 - └─ savings_high_level.spec
 - └─ savings_state_transitions.spec
 - └─ savings_unit_tests.spec
 - └─ savings_valid_state.spec
 - └─ savings_variable_transitions.spec
 - └─ setup/ # Shared setup: ghosts, hooks, summaries
 - └─ boundaries.spec # Loop/ERC20 user count bounds
 - └─ collateral_manager.spec # Per-collateral manager ghost summaries
 - └─ debug/ # Sanity / layout self-test specs
 - └─ env.spec # Environment constraints (timestamp, sender, value)
 - └─ erc20/ # ERC20 CVL models
 - └─ erc20.spec # Ghost-based ERC20 token model
 - └─ safe_erc20.spec # SafeERC20 wrapper summaries
 - └─ token_p.spec # TokenP mint/burn CVL summaries
 - └─ flashParallelToken.spec # FlashParallelToken setup and ghosts
 - └─ helper.spec # CVL helper (assertOnFailure)
 - └─ math.spec # Math library summaries (mulDiv, sqrt)

— parallelizer.spec	# Diamond setup, oracle/manager/whitelist summaries
— parallelizer_boundaries.spec	# Parallelizer verification bounds
— parallelizer_redeemer.spec	# Redeemer facet setup (redeem summaries)
— parallelizer_setter.spec	# Setter facet setup (setOracle exclusion)
— parallelizer_storage.spec	# Storage layout ghosts and Sload/Sstore hooks
— parallelizer_surplus.spec	# Surplus facet setup
— parallelizer_swapper.spec	# Swapper facet setup
— savings.spec	# Savings (ERC4626) setup and ghosts

Assumptions

Formal verification requires assumptions about the code and its environment to address prover timeouts, tool limitations, and state consistency. However, incorrect assumptions can mask real bugs by excluding reachable states from analysis. To maintain transparency, all assumptions are categorized into four groups: **Safe** (real-world constraints that don't reduce security coverage), **Proved** (formally verified invariants reused as preconditions), **Trusted** (initialization state and admin-configured parameters assumed correct because initialization logic is excluded from verification), and **Unsafe** (scope reductions necessary for tractability that may exclude valid scenarios).

Safe Assumptions

These reflect real-world constraints that don't impact security guarantees. In the codebase, every `require` statement that constitutes a safe assumption is annotated with a `"SAFE: ..."` message string.

Environment Constraints ([env.spec](#)):

- The contract does not handle native ETH, so transactions carry no value (`msg.value == 0`)
- The sender is never the zero address and never the contract itself
- Block timestamps fall within realistic bounds (`[65535, max_uint32]`)

Address Separation ([env.spec](#), [parallelizer.spec](#)):

- TokenP is not the Parallelizer and not a collateral address
- For managed collateral, the diamond balance is zero (all funds held by external strategy)

Math Summaries ([math.spec](#)):

- PercentageMathLib's `percentMul` replaced with exact CVL arithmetic: `(x * pct + 5000) / 10000`

ERC-20 Model ([erc20.spec](#)):

- Total supply equals the sum of all individual balances (standard ERC-20 solvency invariant)

Proved Assumptions

These properties have been formally verified as valid state invariants and are used as trusted preconditions (via `requireInvariant`) in state transition and high-level rules. See [Valid State](#) for detailed descriptions and verification statuses.

Parallelizer Structural Invariants ([parallelizer_valid_state.spec](#)):

- Collateral list contains no zero addresses and no duplicates, and list membership is linked to non-zero decimals (VS-P-01, VS-P-02, VS-P-03)
- Non-listed addresses have zero collateral data; stored decimals match ERC-20 decimals for active collaterals (VS-P-24, VS-P-25)
- Boolean flags (`isRedemptionLive`, `isTrusted`, `isSellerTrusted`, `isManaged`, `isMintLive`, `isBurnLive`, `onlyWhitelisted`) are always 0 or 1 (VS-P-08 through VS-P-14)
- Normalizer stays within `[BASE_18, BASE_36]`; slippage tolerance $\leq$ `BASE_9`; surplus buffer ratio is 0 or $\geq$ `BASE_9`; `lastReleasedAt` $\leq$ `block.timestamp` (VS-P-04 through VS-P-07)

Parallelizer Accounting Invariants ([parallelizer_valid_state.spec](#)):

- Global `normalizedStables` equals the sum of per-collateral `normalizedStables` (VS-P-15)
- Total shares equals the sum of per-payee shares; payees are distinct with positive shares; non-payee addresses have zero shares (VS-P-21, VS-P-22, VS-P-23, VS-P-26)

Parallelizer Fee Curve Invariants ([parallelizer_valid_state.spec](#)):

- Mint/burn/redemption fee curve X and Y arrays have matching lengths (VS-P-16, VS-P-17, VS-P-18)
- Mint fee X-array is strictly increasing with $X[0] \neq 0$; burn fee X-array is strictly decreasing with $X[0] \neq \text{BASE_9}$ (VS-P-19, VS-P-20, VS-P-30, VS-P-31)
- Y-values are bounded: $\text{mint} \leq \text{BASE_12}$, $\text{burn} \leq \text{BASE_9}$, $\text{redemption} \in [0, \text{BASE_9}]$; redemption X bounded by BASE_9 (VS-P-27, VS-P-28, VS-P-29, VS-P-32)

Savings Invariants ([savings_valid_state.spec](#)):

- Paused flag and trusted updater flag are always 0 or 1 (VS-SA-01, VS-SA-02)
- lastUpdate never exceeds block.timestamp (VS-SA-03)
- The vault never approves its own shares (VS-SA-04)

FlashParallelToken Invariants ([flashParallelToken_valid_state.spec](#)):

- Fee rate never exceeds 100% (VS-FP-01)

Note: VS-P-14 ([collateralOnlyWhitelistedFlag](#)) had expected violations in [setWhitelistStatus\(\)](#) at the audit commit due to a [real bug](#); it verifies on the mitigated code. It is not used as a [requireInvariant](#) in other rules.

Unsafe Assumptions

These reduce verification scope to make the problem tractable for the prover. In the codebase, every [require](#) statement that constitutes an unsafe assumption is annotated with an "UNSAFE: ..." message string.

External Dependency Summaries ([parallelizer.spec](#), [collateral_manager.spec](#)):

- LibOracle: Replaced with ghost-backed CVL functions. Real oracle calls are not modeled, so price manipulation scenarios are excluded.
- LibManager: Ghost-backed CVL functions model invest/release/totalAssets with per-collateral ghost state. Real strategy interactions are replaced, so strategy failure scenarios are not modeled.
- LibWhitelist: [checkWhitelist](#) summarized as [NONDET](#) — may return true or false. Whitelist bypass scenarios are not modeled.
- LibDiamond: [checkCanCall](#) summarized as [NONDET](#); [isGovernor](#) modeled via persistent ghost mapping. Real access control enforcement is not verified.
- AccessManaged (OpenZeppelin): [canCall](#) and [consumeScheduledOp](#) summarized as [NONDET](#). Access manager restrictions are not verified.
- Initializable/UUPSUpgradeable: Functions [NONDET DELETE](#)-d. Initialization and upgrade logic excluded from verification.

Redeemer Internals ([parallelizer_redeemer.spec](#)):

- [Redeemer.distributeRedemption](#) summarized as [NONDET](#) — the collateral output transfers of [redeem/redeemWithForfeit](#) are not modeled in the parametric Redeemer rules (the TokenP burn accounting, which these rules observe, is fully modeled). Needed because the heavy transfer/release/whitelist loop defeats the prover's points-to analysis.
- [LibAuthorization.computeRedeemNonce](#) summarized as [NONDET](#) — the nonce does not affect any modeled state.
- EIP-3009 [receiveWithAuthorization](#) / [_executeAuthorization](#) are summarized as the exact ERC-20 pull the contract enforces ([value == amount](#), checked at [Redeemer.sol:248](#)) — a faithful model rather than a scope reduction.

Bounded Model ([boundaries.spec](#), [parallelizer_boundaries.spec](#)):

Bounding the number of collaterals, payees, and accounts serves two purposes: it prevents prover timeouts on unbounded quantifiers, and it makes it possible to reason about total supply as a concrete sum of all individual values — enabling protocol-wide conservation invariants such as "global normalizedStables equals the sum of per-collateral normalizedStables."

- Loop iterations capped at 2 across all configurations
- Collateral list length bounded at 2 for universal quantifier tractability
- Payees list length bounded at 2 for universal quantifier tractability
- ERC-20 accounts tracked for at most 5 addresses per token
- Two Savings rules bound vault magnitudes to keep the non-linear [mulDiv](#) in share/asset conversion SMT-tractable: [depositDoesNotDiluteShares](#) ($\leq 2^{128}$) and [withdrawAboveMaxReverts](#) ($\leq 2^{64}$) — the same bounded-magnitude methodology as the [convertTo*](#) rules ([max_uint96](#) / 10^{31})

Trusted Assumptions

These assume that initialization has been completed and that admin-configured parameters fall within reasonable operational bounds. Since `initialize()` and upgrade logic are excluded from verification (summarized as `NONDET DELETE`), the post-initialization state is taken as a precondition. In the codebase, every `require` statement that constitutes a trusted assumption is annotated with a `"TRUSTED: ..."` message string.

Initialization State ([parallelizer.spec](#)):

- `accessManager != 0` — set by `initialize()`; all access control checks depend on this being non-zero
- `statusReentrant ∈ {1, 2}` — set by the first `nonReentrant` call; the reentrancy guard uses transient storage (EIP-1153) which the prover does not natively model, so the guard state is constrained to its two valid values via ghost variables

Oracle Value Bounds ([parallelizer.spec](#)):

- All oracle ghost functions (`readMint`, `readBurn`, `getBurnOracle`, `readRedemption`) return values in `(0, max_uint128]`
- Burn ratio and minRatio bounded to `(0, BASE_18]`
- These bounds assume the admin has configured a functioning oracle that returns non-zero, non-overflowing prices

Savings Rate Bounds ([savings.spec](#)):

- Savings rate bounded to `[1012, 1021]` — prevents solver overflow in `_computeUpdatedAssets` rate exponentiation; the admin is trusted to set rates within this range (fuzz tests use `[1014, 1020]`)

Verification Properties

Links to specific CVL spec files are provided for each property. Each property has two status columns:

- **Audit** -- status at the audit commit
- **Mitig** -- status after the mitigation review: the full suite was re-run against the mitigated code — all 490 rule/invariant runs verify (✅ 490 / ❌ 0; the shared Parallelizer rules run once per facet)

Status indicators:

- ✅ Verified
- ❌ Violated
- — Not applicable at the audit commit

Note: the mitigation PRs also changed several public APIs — Parallelizer Setter `togglePause*` → `pauseMint/unpauseMint`, `pauseBurn/unpauseBurn`, `pauseRedeem/unpauseRedeem`; Savings `togglePause` → `pause/unpause`, `estimatedAPR` → `estimatedAPY`, and `setMaxRate` now clamps the active rate down to the new cap. Affected rules were renamed or split into `a/b` variants accordingly. For renamed rules with unchanged semantics the **Audit** column shows the predecessor rule's status; rules marked `—` check post-fix behaviour that did not exist at the audit commit.

Valid State

Valid State properties define the fundamental invariants that must always hold true throughout the protocol's lifecycle. These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Parallelizer

The 32 Parallelizer invariants are verified independently across all four facets (Swapper, Redeemer, Setter, Surplus).

Property	Name	Description	Audit	Mitig	Notes
VS-P-01	<code>collateralListNoZeroValues</code>	Collateral list entries are non-zero addresses <code>forall i: i < listLen => collateralList[i] != 0</code>	✅	✅	
VS-P-02	<code>collateralListNoDuplicates</code>	Collateral list entries are distinct <code>forall i, j: i < listLen AND j < listLen AND i != j => collateralList[i] != collateralList[j]</code>	✅	✅	
VS-P-03	<code>collateralListDecimalsNonZero</code>	Collateral list linked to mapping (decimals non-zero) <code>forall i: i < listLen <=> decimals[collateralList[i]] != 0</code>	✅	✅	
VS-P-04	<code>normalizerBounded</code>	Normalizer stays within [BASE_18, BASE_36] <code>BASE_18 <= norm <= BASE_36</code>	✅	✅	
VS-P-05	<code>slippageToleranceBounded</code>	Slippage tolerance per collateral ≤ BASE_9 <code>forall a: slippageTolerance[a] <= BASE_9</code>	✅	✅	
VS-P-06	<code>surplusBufferRatioBounded</code>	Surplus buffer ratio is 0 or ≥ BASE_9 <code>surplusBufferRatio == 0 OR surplusBufferRatio >= BASE_9</code>	✅	✅	
VS-P-07	<code>lastReleasedAtBounded</code>	<code>lastReleasedAt ≤ block.timestamp</code> <code>lastReleasedAt <= block.timestamp</code>	✅	✅	
VS-P-08	<code>isRedemptionLiveFlag</code>	<code>isRedemptionLive</code> is boolean {0, 1} <code>isRedemptionLive in {0, 1}</code>	✅	✅	
VS-P-09	<code>isTrustedFlag</code>	<code>isTrusted</code> mapping values are boolean {0, 1} <code>forall a: isTrusted[a] in {0, 1}</code>	✅	✅	
VS-P-10	<code>isSellerTrustedFlag</code>	<code>isSellerTrusted</code> mapping values are boolean {0, 1} <code>forall a: isSellerTrusted[a] in {0, 1}</code>	✅	✅	
VS-P-11	<code>collateralIsManagedFlag</code>	<code>isManaged</code> is boolean {0, 1} <code>forall t: isManaged[t] in {0, 1}</code>	✅	✅	
VS-P-12	<code>collateralIsMintLiveFlag</code>	<code>isMintLive</code> is boolean {0, 1} <code>forall t: isMintLive[t] in {0, 1}</code>	✅	✅	
VS-P-13	<code>collateralIsBurnLiveFlag</code>	<code>isBurnLive</code> is boolean {0, 1} <code>forall t: isBurnLive[t] in {0, 1}</code>	✅	✅	

VS-P-14	<code>collateralOnlyWhitelistedFlag</code>	onlyWhitelisted is boolean {0, 1} <code>forall t: onlyWhitelisted[t] in {0, 1}</code>	✗	✓	At audit: violated in <code>setWhitelistStatus()</code> — see Real Issue #3 ; verifies after the fix
VS-P-15	<code>normalizedStablesSumConservation</code>	Global normalizedStables == sum of per-collateral normalizedStables <code>NS == sum(collatNS[t] for t in collateralList)</code>	✓	✓	
VS-P-16	<code>feeMintLengthMatching</code>	Mint fee curve X and Y arrays have same length <code>forall t: len(xFeeMint[t]) == len(yFeeMint[t])</code>	✓	✓	
VS-P-17	<code>feeBurnLengthMatching</code>	Burn fee curve X and Y arrays have same length <code>forall t: len(xFeeBurn[t]) == len(yFeeBurn[t])</code>	✓	✓	
VS-P-18	<code>redemptionCurveLengthMatching</code>	Redemption curve X and Y arrays have same length <code>len(xRedemptionCurve) == len(yRedemptionCurve)</code>	✓	✓	
VS-P-19	<code>feeMintXOrdering</code>	Mint fee curve X-array strictly increasing <code>forall t: len(xFeeMint[t]) == 2 => xFeeMint[t][0] < xFeeMint[t][1]</code>	✓	✓	
VS-P-20	<code>feeBurnXOrdering</code>	Burn fee curve X-array strictly decreasing <code>forall t: len(xFeeBurn[t]) == 2 => xFeeBurn[t][0] > xFeeBurn[t][1]</code>	✓	✓	
VS-P-21	<code>totalSharesSumConservation</code>	totalShares == sum of shares over payees <code>totalShares == sum(shares[payees[i]] for i < payeesLen)</code>	✓	✓	
VS-P-22	<code>payeesNoDuplicates</code>	Payees array has no duplicates <code>forall i, j: i < payeesLen AND j < payeesLen AND i != j => payees[i] != payees[j]</code>	✓	✓	
VS-P-23	<code>payeeSharesPositive</code>	Every active payee has positive shares <code>forall i: i < payeesLen => shares[payees[i]] > 0</code>	✓	✓	
VS-P-24	<code>collateralDecimalsMatchErc20</code>	Stored decimals matches ERC20 decimals for active collaterals <code>forall i: i < listLen => decimals[collateralList[i]] == erc20Decimals[collateralList[i]]</code>	✓	✓	
VS-P-25	<code>collateralDataZeroOutsideList</code>	Non-listed addresses have zero collateral data <code>forall a: a not in collateralList <=> allCollatFields[a] == 0</code>	✓	✓	
VS-P-26	<code>sharesZeroOutsidePayees</code>	Non-payee addresses have zero shares <code>forall a: a not in payees => shares[a] == 0</code>	✓	✓	
VS-P-27	<code>feeMintYBounded</code>	Mint fee Y-values weakly increasing and bounded by BASE_12 <code>forall t: yFeeMint[t][0] <= yFeeMint[t][1] <= BASE_12</code>	✓	✓	
VS-P-28	<code>feeBurnYBounded</code>	Burn fee Y-values weakly increasing and bounded by BASE_9 <code>forall t: yFeeBurn[t][0] <= yFeeBurn[t][1] <= BASE_9</code>	✓	✓	
VS-P-29	<code>redemptionCurveYBounded</code>	Redemption curve Y-values bounded in [0, BASE_9] <code>forall i: i < len(yRedemptionCurve) => 0 <= yRedemptionCurve[i] <= BASE_9</code>	✓	✓	
VS-P-30	<code>feeMintXAnchor</code>	<code>xFeeMint[0] == 0</code> <code>forall t: len(xFeeMint[t]) > 0 => xFeeMint[t][0] == 0</code>	✓	✓	
VS-P-31	<code>feeBurnXAnchor</code>	<code>xFeeBurn[0] == BASE_9</code> <code>forall t: len(xFeeBurn[t]) > 0 => xFeeBurn[t][0] == BASE_9</code>	✓	✓	
VS-P-32	<code>redemptionCurveXBounded</code>	Redemption curve X bounded by BASE_9 <code>forall i: i < len(xRedemptionCurve) => xRedemptionCurve[i] <= BASE_9</code>	✓	✓	

Savings

Property	Name	Description	Audit	Mitig	Notes
VS-SA-01	<code>pausedFlagIsZeroOrOne</code>	Paused flag is always 0 or 1 <code>paused == 0 OR paused == 1</code>	✓	✓	
VS-SA-02	<code>trustedUpdaterFlagIsZeroOrOne</code>	Trusted updater mapping values are always 0 or 1 <code>forall a: isTrustedUpdater[a] == 0 OR isTrustedUpdater[a] == 1</code>	✓	✓	
VS-SA-03	<code>lastUpdateNeverExceedsBlockTime</code>	lastUpdate never exceeds current block timestamp <code>lastUpdate <= block.timestamp</code>	✓	✓	
VS-SA-04	<code>vaultNeverApprovesOwnShares</code>	The vault itself never has share allowances granted <code>forall spender: allowance[vault][spender] == 0</code>	✓	✓	

FlashParallelToken

Property	Name	Description	Audit	Mitig	Notes
VS-FP-01	<code>feeRateNeverExceedsMax</code>	Fee rate never exceeds 100% (PERCENTAGE_FACTOR) <code>forall token: feesRate[token] <= PERCENTAGE_FACTOR</code>	✓	✓	

Variable Transitions

Variable Transition properties verify that specific storage variables change only under expected conditions (monotonicity, set-once semantics, or authorized-function restrictions). These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Parallelizer

Property	Name	Description	Audit	Mitig	Notes
VT-P-01	<code>tokenPNeverChanges</code>	tokenP is immutable after initialization <code>forall f: tokenP' == tokenP</code>	✓	✓	
VT-P-02	<code>lastReleasedAtOnlyIncreases</code>	lastReleasedAt only increases <code>forall f: lastReleasedAt' >= lastReleasedAt</code>	✓	✓	
VT-P-03	<code>collateralDecimalsZeroNonZeroTransition</code>	Collateral decimals only transition between zero and non-zero <code>forall f, t: decimals[t]' != decimals[t] => (decimals[t] == 0 OR decimals[t]' == 0)</code>	✓	✓	

Savings

Property	Name	Description	Audit	Mitig	Notes
VT-SA-01	<code>assetAddressNeverChanges</code>	Asset address is immutable once initialized <pre>forall f: asset != 0 => asset' == asset</pre>	✓	✓	
VT-SA-02	<code>lastUpdateOnlyAdvancesForward</code>	lastUpdate timestamp only advances forward <pre>forall f: lastUpdate' >= lastUpdate</pre>	✓	✓	
VT-SA-03	<code>rateOnlyChangesViaSetRate</code>	Rate only changes via setRate or setMaxRate <pre>forall f: rate' != rate => f in {setRate, setMaxRate}</pre>	✓	✓	post-fix <code>setMaxRate</code> clamps rate down to the new cap, so it is a second legitimate writer
VT-SA-04	<code>pausedOnlyChangesViaPauseUnpause</code>	Paused only changes via pause/unpause <pre>forall f: paused' != paused => f in {pause, unpause}</pre>	✓	✓	

FlashParallelToken

Property	Name	Description	Audit	Mitig	Notes
VT-FP-01	<code>feeRecipientCannotBeResetToZero</code>	Fee recipient cannot be set back to zero once assigned <pre>forall f: feeRecipient != 0 => feeRecipient' != 0</pre>	✓	✓	

State Transitions

State Transition properties verify the correctness of transitions between valid states. These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Parallelizer

Property	Name	Description	Audit	Mitig	Notes
ST-P-01	<code>renormalizationResetsNormalizerToBase27</code>	When both normalizer and normalizedStables change, normalizer resets to BASE_27 <pre>forall f: norm' != norm AND NS' != NS => norm' == BASE_27</pre>	✓	✓	
ST-P-02	<code>perCollateralNsChangeImpliesGlobalNsChange</code>	Per-collateral normalizedStables change implies global change <pre>forall f, t: collatNS[t]' != collatNS[t] => NS' != NS</pre>	✓	✓	
ST-P-03	<code>globalNsChangeImpliesSomeCollatNsChange</code>	Global normalizedStables change implies at least one per-collateral change <pre>forall f: NS' != NS => exists t in collateralList: collatNS[t]' != collatNS[t]</pre>	✓	✓	
		Multiple per-collateral changes imply renormalization <pre>forall f: collatNS[t0]'</pre>			

ST-P-04	<code>multipleCollateralNsChangesImpliesRenormalization</code>	<code>!= collatNS[t0] AND collatNS[t1] != collatNS[t1] => norm' == BASE_27</code>	✓	✓	
ST-P-05	<code>collateralDecimalsAndListAtomicity</code>	Decimals set implies list grew; cleared implies list shrank <code>forall f, t: (decimals[t]==0 AND decimals[t]!=0 => listLen'>listLen) AND (decimals[t]!=0 AND decimals[t]==0 => listLen'<listLen)</code>	✓	✓	
ST-P-06	<code>collateralListGrowthPreservesTokenPSupply</code>	Adding a collateral does not change tokenP totalSupply <code>forall f: listLen' > listLen => supply' == supply</code>	✓	✓	
ST-P-07	<code>tokenPSupplyChangeImpliesProtocolStateChange</code>	TokenP supply change implies NS, normalizer, or surplus release changed — up to the documented dust no-op tolerance <code>forall f: supply' != supply => NS' != NS OR norm' != norm OR lastReleasedAt' == block.timestamp OR (supply - supply')*BASE_27 < norm OR (supply - supply')*BASE_27 < NS</code>	✗	✓	At audit: violated at dust amounts (integer- division rounding in <code>swapExactInput()</code> / <code>swapExactOutput()</code>); the rule lacked tolerance disjuncts. Refined to the documented dust no-op characterization; verifies with redeem fully covered
ST-P-08	<code>stablecoinCapEnforcedOnNsIncrease</code>	Per-collateral issued-value increase respects stablecoinCap <code>forall f, t: collatNS[t]'*norm' > collatNS[t]*norm AND cap[t] > 0 => collatNS[t]'*norm'/BASE_27 <= cap[t]</code>	✗	✓	At audit: violated in <code>adjustStablecoins</code> — see Real Issue #4 — and at dust redeem amounts. Refined to trigger on the implied issued value <code>collatNS*normalizer</code> (preserved by renormalization); <code>updateNormalizer</code> (uncapped interest accrual by design) excluded at its Redeemer use-site
ST-P-09	<code>tokenPSupplyIncreaseImpliesNsIncrease</code>	TokenP supply increase implies normalizedStables increase <code>forall f: supply' > supply => NS' > NS</code>	✓	✓	
ST-P-10	<code>diamondTokenPBalanceChangeImpliesSurplusOrRecoveryPath</code>	Diamond tokenP balance change implies surplus path or recovery function <code>forall f: bal[tokenP] [diamond]' != bal[tokenP] [diamond] => NS' != NS OR lastReleasedAt' == block.timestamp OR f.selector == recoverERC20.selector</code>	✓	✓	
ST-P-11	<code>accountingChangePreservesConfiguration</code>	Accounting change preserves configuration parameters <code>forall f: NS' != NS OR norm' != norm => config' == config</code>	✓	✓	
ST-P-12	<code>lastReleasedAtOnlyUpdatesToTimestamp</code>	<code>lastReleasedAt</code> only updates to <code>block.timestamp</code> <code>forall f: lastReleasedAt' != lastReleasedAt => lastReleasedAt' == block.timestamp</code>	✓	✓	
ST-P-13	<code>collateralListShrinkImpliesZeroNs</code>	Removed collateral must have had zero normalizedStables <code>forall f, t: listLen' < listLen AND decimals[t]!=0 AND decimals[t]==0 => collatNS[t] == 0</code>	✓	✓	

Savings

Property	Name	Description	Audit	Mitig	Notes
ST-SA-01	<code>stateChangeAlwaysRefreshesTimestamp</code>	Any state change forces lastUpdate to block.timestamp <code>forall f: (rate' != rate OR supply' != supply) => lastUpdate' == block.timestamp</code>	✓	✓	
ST-SA-02	<code>pausedContractCannotChangeShareSupply</code>	Share total supply cannot change while paused <code>forall f: paused == 1 => supply' == supply</code>	✓	✓	
ST-SA-03	<code>rateCannotIncreaseAboveMaxRate</code>	Rate can only increase up to maxRate <code>forall f: rate' > rate => rate' <= maxRate'</code>	✓	✓	

FlashParallelToken

Property	Name	Description	Audit	Mitig	Notes
ST-FP-01	<code>tokensOnlyWithdrawnToFeeRecipient</code>	Tokens can only leave the contract to the fee recipient <code>forall f: bal[t][contract]' < bal[t][contract] => bal[t][feeRecipient]' - bal[t][feeRecipient] == bal[t][contract] - bal[t][contract]'</code>	✓	✓	
ST-FP-02	<code>feeRecipientAndTokenSettingsChangeSeparately</code>	Fee recipient and token settings cannot change in the same call <code>forall f: feeRecipient' != feeRecipient => (feesRate[t]' == feesRate[t] AND maxBorrowable[t]' == maxBorrowable[t] AND isActive[t]' == isActive[t])</code>	✓	✓	
ST-FP-03	<code>tokenSettingsDoNotAffectOtherTokens</code>	Changing one token's settings does not affect other tokens <code>forall f, tA != tB: config[tA]' != config[tA] => config[tB]' == config[tB]</code>	✓	✓	
ST-FP-04	<code>configChangesDoNotMoveTokens</code>	Config changes do not move tokens or change supply <code>forall f: configChanged => bal[t][acct]' == bal[t][acct] AND supply[t]' == supply[t]</code>	✓	✓	

High-Level

High-Level properties verify multi-step business logic, user isolation across core operations, and monotonicity of core operations.

Parallelizer — Swapper

Property	Name	Description	Audit	Mitig	Notes
HL-SW-01	<code>mintExactInputSupplyDelta</code>	Mint via swapExactInput increases TokenP totalSupply by exactly amountOut <code>amountIn > 0 AND amountOut > 0 => supply' == supply + amountOut</code>	✓	✓	
HL-SW-		Burn via swapExactInput decreases TokenP totalSupply by exactly amountIn			

02	burnExactInputSupplyDelta	amountIn > 0 AND amountOut > 0 => supply' == supply - amountIn	✓	✓	
HL-SW-03	mintExactOutputSupplyDelta	Mint via swapExactOutput increases TokenP totalSupply by exactly amountOut amountIn > 0 AND amountOut > 0 => supply' == supply + amountOut	✓	✓	
HL-SW-04	burnExactOutputSupplyDelta	Burn via swapExactOutput decreases TokenP totalSupply by returned amountIn amountIn > 0 AND amountOut > 0 => supply' == supply - amountIn	✓	✓	
HL-SW-05	mintCollateralTransfer	Mint transfers exactly amountIn of collateral from sender to diamond amountIn > 0 AND amountOut > 0 => bal[collat][sender]' == bal[collat][sender] - amountIn AND bal[collat][diamond]' == bal[collat][diamond] + amountIn	✓	✓	
HL-SW-06	burnCollateralTransfer	Burn transfers exactly amountOut of collateral from diamond to recipient amountIn > 0 AND amountOut > 0 => bal[collat][diamond]' == bal[collat][diamond] - amountOut AND bal[collat][to]' == bal[collat][to] + amountOut	✓	✓	
HL-SW-07	swapOnlyAffectsTargetCollateral	Swap only affects target collateral's normalizedStables NOT (collatNS[c0]' != collatNS[c0] AND collatNS[c1]' != collatNS[c1])	✓	✓	
HL-SW-08	zeroAmountInNoOp	Zero amountIn produces zero output and no state changes amountIn == 0 => amountOut == 0 AND NS' == NS AND supply' == supply	✓	✓	
HL-SW-09	mintExactInputIncreasesNs	Mint via swapExactInput strictly increases normalizedStables amountIn > 0 AND amountOut > 0 => collatNS[t]' > collatNS[t] AND NS' > NS	✓	✓	
HL-SW-10	burnExactInputDecreasesNs	Burn via swapExactInput does not increase normalizedStables amountIn > 0 AND amountOut > 0 => collatNS[t]' <= collatNS[t] AND NS' <= NS	✓	✓	
HL-SW-11	mintExactOutputIncreasesNs	Mint via swapExactOutput strictly increases normalizedStables amountIn > 0 AND amountOut > 0 => collatNS[t]' > collatNS[t] AND NS' > NS	✓	✓	
HL-SW-12	burnExactOutputDecreasesNs	Burn via swapExactOutput does not increase normalizedStables amountIn > 0 AND amountOut > 0 => collatNS[t]' <= collatNS[t] AND NS' <= NS	✓	✓	
HL-SW-13	swapExactInputSlippageGuarantee	amountOut ≥ amountOutMin on success !REVERTS => amountOut >= amountOutMin	✓	✓	
HL-SW-14	swapExactOutputSlippageGuarantee	amountIn ≤ amountInMax on success !REVERTS => amountIn <= amountInMax	✓	✓	
HL-SW-		swapExactOutput with zero amountOut is a no-op			

15	<code>zeroAmountOutExactOutputNoOp</code>	<code>amountOut == 0 => amountIn == 0 AND NS' == NS AND supply' == supply</code>	✓	✓	
HL-SW-16	<code>quoteInMatchesMintExactInput</code>	quoteIn matches swapExactInput for mint direction <code>quoteIn(amountIn, collat, tokenP) == swapExactInput(amountIn, collat, tokenP)</code>	✓	✓	
HL-SW-17	<code>quoteInMatchesBurnExactInput</code>	quoteIn matches swapExactInput for burn direction <code>quoteIn(amountIn, tokenP, collat) == swapExactInput(amountIn, tokenP, collat)</code>	✓	✓	
HL-SW-18	<code>quoteInSameForAnyCaller</code>	quoteIn returns same value regardless of caller <code>forall sender1, sender2: quoteIn(sender1, ...) == quoteIn(sender2, ...)</code>	✓	✓	
HL-SW-19	<code>quoteOutSameForAnyCaller</code>	quoteOut returns same value regardless of caller <code>forall sender1, sender2: quoteOut(sender1, ...) == quoteOut(sender2, ...)</code>	✓	✓	

Parallelizer — Redeemer

Property	Name	Description	Audit	Mitig	Notes
HL-RD-01	<code>updateNormalizerIncreaseGrowsStablecoinsIssued</code>	increase=true does not decrease stablecoinsIssued <code>updateNormalizer(amount, true) => stablecoinsIssued' >= stablecoinsIssued</code>	✓	✓	
HL-RD-02	<code>updateNormalizerDecreaseShrinksStablecoinsIssued</code>	increase=false does not increase stablecoinsIssued <code>updateNormalizer(amount, false) => stablecoinsIssued' <= stablecoinsIssued</code>	✓	✓	
HL-RD-03	<code>updateNormalizerIncreaseRaisesNormalizer</code>	Without renorm, increase=true raises normalizer <code>updateNormalizer(amount, true) AND NS' = NS => norm' >= norm</code>	✓	✓	
HL-RD-04	<code>updateNormalizerDecreaseLowersNormalizer</code>	Without renorm, increase=false lowers normalizer <code>updateNormalizer(amount, false) AND NS' = NS => norm' <= norm</code>	✓	✓	
HL-RD-05	<code>redeemDecreasesStablecoinsIssued</code>	After redeem, stablecoinsIssued ≤ before <code>redeem(amount, ...) => stablecoinsIssued' <= stablecoinsIssued</code>	✓	✓	
HL-RD-06	<code>largerRedeemDecreasesStablecoinsIssuedMore</code>	Larger redeem decreases stablecoinsIssued at least as much <code>amount1 >= amount2 => stablecoinsIssued'(redeem(amount1)) <= stablecoinsIssued'(redeem(amount2))</code>	✓	✓	
HL-RD-07	<code>largerUpdateNormalizerIncreasesMore</code>	Larger amount produces larger stablecoinsIssued increase <code>amount1 >= amount2 => stablecoinsIssued'(updateNormalizer(amount1, true)) >= stablecoinsIssued'(updateNormalizer(amount2, true))</code>	✓	✓	
HL-RD-08	<code>updateNormalizerIncreaseBoundedByAmount</code>	Increase in stablecoinsIssued bounded by amount <code>updateNormalizer(amount, true) => stablecoinsIssued' - stablecoinsIssued <= amount</code>	✓	✓	
HL-RD-09	<code>updateNormalizerDecreaseBoundedByAmount</code>	Decrease in stablecoinsIssued bounded by amount + renorm rounding <code>updateNormalizer(amount, false) => stablecoinsIssued - stablecoinsIssued' <= amount</code>	✓	✓	
HL-RD-10	<code>redeemWithoutRenormPreservesNormalizedStables</code>	If normalizer ≠ BASE_27 after redeem, normalizedStables unchanged <code>redeem(amount, ...) AND norm' != BASE_27 => NS' = NS</code>	✓	✓	

Parallelizer — Surplus

Property	Name	Description	Audit	Mitig	Notes
HL-S-01	<code>processSurplusIncreasesTokenPSupply</code>	processSurplus increases TokenP totalSupply by exactly issuedAmount <code>supply' == supply + issuedAmount</code>	✓	✓	
HL-S-02	<code>processSurplusCreditsTokenPToDiamond</code>	processSurplus credits exactly issuedAmount to diamond <code>bal[TokenP][diamond]' == bal[TokenP][diamond] + issuedAmount</code>	✓	✓	
HL-S-03	<code>releaseDistributesAllTokenP</code>	release does not increase diamond TokenP balance <code>bal[TokenP][diamond]' <= bal[TokenP][diamond]</code>	✓	✓	
HL-S-04	<code>processSurplusIncreasesNormalizedStables</code>	processSurplus does not decrease global normalizedStables <code>NS' >= NS</code>	✓	✓	
HL-S-05	<code>processSurplusIncreasesPerCollateralNs</code>	processSurplus does not decrease per-collateral normalizedStables <code>collatNS[collateral]' >= collatNS[collateral]</code>	✓	✓	
HL-S-06	<code>processSurplusPositiveIssuedImpliesNsIncrease</code>	Positive issuedAmount implies strict normalizedStables increase <code>issuedAmount > 0 => NS' > NS</code>	✓	✓	
HL-S-07	<code>surplusLifecycleProcessThenRelease</code>	processSurplus then release: balance does not increase <code>bal[TokenP][diamond] == 0 => bal[TokenP][diamond]_afterRelease <= bal[TokenP][diamond]_afterProcess</code>	✓	✓	
HL-S-08	<code>processSurplusCapsToMaxCollateral</code>	collateralSurplus capped to maxCollateralAmount <code>maxCollateralAmount > 0 => collateralSurplus <= maxCollateralAmount</code>	✓	✓	
HL-S-09	<code>processSurplusReachableWithHighBufferRatio</code>	Reachable when surplusBufferRatio > 100% <code>satisfy(surplusBufferRatio > BASE_9 AND collateralSurplus > 0 AND issuedAmount > 0)</code>	✗	✓	At audit: violated — see Real Issue #2 ; verifies after the fix
HL-S-10	<code>processSurplusManagedCollateralIsReachable</code>	Reachable for managed collateral <code>satisfy(isManaged[collateral] > 0 AND collateralSurplus > 0 AND issuedAmount > 0)</code>	✗	✓	At audit: violated — see Real Issue #1 ; verifies after the fix

Parallelizer — Setter

Property	Name	Description	Audit	Mitig	Notes
HL-SET-01	<code>pauseUnpauseMintRoundTrip</code>	pauseMint then unpauseMint restores isMintLive <code>pauseMint; unpauseMint => isMintLive' == isMintLive</code>	✓	✓	
HL-SET-02	<code>pauseUnpauseBurnRoundTrip</code>	pauseBurn then unpauseBurn restores isBurnLive <code>pauseBurn; unpauseBurn => isBurnLive' == isBurnLive</code>	✓	✓	
HL-SET-03	<code>pauseUnpauseRedeemRoundTrip</code>	pauseRedeem then unpauseRedeem restores isRedemptionLive <code>pauseRedeem; unpauseRedeem => isRedemptionLive' == isRedemptionLive</code>	✓	✓	
HL-SET-04	<code>toggleTrustedUpdaterRoundTrip</code>	toggleTrusted(Updater) twice restores isTrusted <code>toggleTrustedUpdater(a); toggleTrustedUpdater(a) => isTrusted'[a] == isTrusted[a]</code>	✓	✓	
HL-SET-05	<code>toggleTrustedSellerRoundTrip</code>	toggleTrusted(Seller) twice restores isSellerTrusted <code>toggleTrustedSeller(a); toggleTrustedSeller(a) => isSellerTrusted'[a] == isSellerTrusted[a]</code>	✓	✓	
HL-SET-06	<code>adjustStablecoinsRoundTrip</code>	adjustStablecoins increase then decrease restores normalizedStables <code>adjustStablecoins(t, amt, true); adjustStablecoins(t, amt, false) => NS' == NS AND collatNS'[t] == collatNS[t]</code>	✓	✓	
HL-SET-07	<code>adjustStablecoinsIncreaseMonotonicity</code>	adjustStablecoins with increase=true does not decrease normalizedStables <code>amt > 0 => adjustStablecoins(t, amt, true) => NS' >= NS AND collatNS'[t] >= collatNS[t]</code>	✓	✓	
HL-SET-08	<code>adjustStablecoinsDecreaseMonotonicity</code>	adjustStablecoins with increase=false does not increase normalizedStables <code>amt > 0 => adjustStablecoins(t, amt, false) => NS' <= NS AND collatNS'[t] <= collatNS[t]</code>	✓	✓	
HL-SET-09	<code>adjustStablecoinsGlobalLocalDeltaConservation</code>	adjustStablecoins per-collateral delta equals global delta <code>adjustStablecoins(t, amt, inc) => NS' - NS == collatNS'[t] - collatNS[t]</code>	✓	✓	

Savings

Property	Name	Description	Audit	Mitig	Notes
HL-SA-01	<code>depositThenRedeemCannotProfit</code>	Depositing then redeeming yields at most the original assets <code>redeem(deposit/assets)) <= assets</code>	✓	✓	
HL-SA-02	<code>mintThenRedeemCannotProfit</code>	Minting then redeeming yields at most the paid assets <code>redeem(shares) <= mint(shares)</code>	✓	✓	
HL-SA-03	<code>depositThenWithdrawCannotProfit</code>	Depositing then withdrawing burns at least the minted shares <code>withdraw/assets).sharesBurned >= deposit/assets).sharesMinted</code>	✓	✓	
HL-SA-04	<code>withdrawThenDepositCannotProfit</code>	Withdrawing then depositing mints at most the burned shares <code>deposit/assets).sharesMinted <= withdraw/assets).sharesBurned</code>	✓	✓	
HL-SA-05	<code>redeemThenMintCannotProfit</code>	Redeeming then minting costs at least the received assets <code>mint(shares) >= redeem(shares)</code>	✓	✓	
HL-SA-06	<code>mintThenWithdrawCannotProfit</code>	Minting then withdrawing burns at least the minted shares <code>withdraw(mint(shares)).sharesBurned >= shares</code>	✓	✓	

HL-SA-07	<code>moreAssetsDepositedMeansMoreShares</code>	Deposit monotonicity: more assets yield at least as many shares <code>assets1 >= assets2 =></code> <code>deposit(assets1).shares >=</code> <code>deposit(assets2).shares</code>	✓	✓	
HL-SA-08	<code>moreSharesRedeemedMeansMoreAssets</code>	Redeem monotonicity: more shares yield at least as many assets <code>shares1 >= shares2 =></code> <code>redeem(shares1).assets >=</code> <code>redeem(shares2).assets</code>	✓	✓	
HL-SA-09	<code>moreSharesMintedCostsMoreAssets</code>	Mint monotonicity: more shares cost at least as many assets <code>shares1 >= shares2 =></code> <code>mint(shares1).assets >=</code> <code>mint(shares2).assets</code>	✓	✓	
HL-SA-10	<code>moreAssetsWithdrawnBurnsMoreShares</code>	Withdraw monotonicity: more assets burn at least as many shares <code>assets1 >= assets2 =></code> <code>withdraw(assets1).shares >=</code> <code>withdraw(assets2).shares</code>	✓	✓	
HL-SA-11	<code>depositDoesNotDiluteShares</code>	Deposit does not decrease share-to-asset exchange rate <code>convertToAssets(1e18)' >=</code> <code>convertToAssets(1e18) after deposit</code>	✓	✓	
HL-SA-12	<code>mintDoesNotDiluteShares</code>	Mint does not decrease share-to-asset exchange rate <code>convertToAssets(1e18)' >=</code> <code>convertToAssets(1e18) after mint</code>	✓	✓	
HL-SA-13	<code>redeemDoesNotDiluteShares</code>	Redeem does not decrease share-to-asset exchange rate <code>convertToAssets(1e18)' >=</code> <code>convertToAssets(1e18) after redeem</code>	✓	✓	
HL-SA-14	<code>withdrawDoesNotDiluteShares</code>	Withdraw does not decrease share-to-asset exchange rate <code>convertToAssets(1e18)' >=</code> <code>convertToAssets(1e18) after withdraw</code>	✓	✓	
HL-SA-15	<code>depositIsAdditive</code>	Splitting a deposit produces total shares within 1 of a single deposit <code>abs(deposit(a+b).shares -</code> <code>(deposit(a).shares +</code> <code>deposit(b).shares)) <= 1</code>	✓	✓	
HL-SA-16	<code>donationDoesNotInflateShareValue</code>	Direct asset transfer to the vault must not change the share-to-asset rate <code>convertToAssets(SHARE_UNIT)' ==</code> <code>convertToAssets(SHARE_UNIT) after</code> <code>donation</code>	✗	✓	At audit: dormant-accrual donation attack — see Real Issue #5 ; verifies after the fix
HL-SA-17	<code>donationDoesNotAmplifyTotalAssets</code>	Donation must not be amplified beyond its face value in totalAssets <code>totalAssets()' - totalAssets() <=</code> <code>donation</code>	✗	✓	At audit: dormant-accrual donation attack — see Real Issue #5 ; verifies after the fix

FlashParallelToken

Property	Name	Description	Audit	Mitig	Notes
HL-FP-01	<code>flashLoanDoesNotChangeTotalSupply</code>	Flash loan mint and burn cancel out, total supply unchanged <code>supply[t]' == supply[t] after flashLoan</code>	✓	✓	
HL-FP-02	<code>flashLoanCollectsExactFee</code>	Contract balance increases by exactly the expected fee <code>bal[t][contract]' == bal[t][contract] + (amount * feesRate[t] + 5000) / 10000 after flashLoan</code>	✓	✓	
HL-FP-03	<code>borrowerCannotProfitFromFlashLoan</code>	Borrower's balance cannot increase after a flash loan <code>bal[t][receiver]' <= bal[t][receiver] after flashLoan</code>	✓	✓	
HL-FP-04	<code>flashLoanFeeNeverExceedsBorrowedAmount</code>	Fee charged never exceeds the borrowed amount <code>bal[t][contract]' - bal[t][contract] <= amount after flashLoan</code>	✓	✓	

Unit Tests

Unit Test properties verify basic behavior of individual functions: revert conditions, direct effects, and non-effects on unrelated state.

Parallelizer — Swapper

Property	Name	Description	Audit	Mitig	Notes
UT-SW-01	<code>swapExactInputRevertsAfterDeadline</code>	Reverts when <code>block.timestamp > deadline</code> <code>block.timestamp > deadline => REVERTS</code>	✓	✓	
UT-SW-02	<code>swapExactOutputRevertsAfterDeadline</code>	Reverts when <code>block.timestamp > deadline</code> <code>block.timestamp > deadline => REVERTS</code>	✓	✓	
UT-SW-03	<code>swapExactInputMintRevertsWhenPaused</code>	Mint reverts when <code>isMintLive == 0</code> <code>isMintLive[collat] == 0 => REVERTS</code>	✓	✓	
UT-SW-04	<code>swapExactInputBurnRevertsWhenPaused</code>	Burn reverts when <code>isBurnLive == 0</code> <code>isBurnLive[collat] == 0 => REVERTS</code>	✓	✓	
UT-SW-05	<code>swapExactOutputMintRevertsWhenPaused</code>	Mint reverts when <code>isMintLive == 0</code> <code>isMintLive[collat] == 0 => REVERTS</code>	✓	✓	
UT-SW-06	<code>swapExactOutputBurnRevertsWhenPaused</code>	Burn reverts when <code>isBurnLive == 0</code> <code>isBurnLive[collat] == 0 => REVERTS</code>	✓	✓	
UT-SW-07	<code>mintExactInputIsReachable</code>	Mint via <code>swapExactInput</code> is reachable with positive output <code>satisfy(swapExactInput(amountIn, collat, tokenP).amountOut > 0)</code>	✓	✓	
UT-SW-08	<code>burnExactInputIsReachable</code>	Burn via <code>swapExactInput</code> is reachable with positive output <code>satisfy(swapExactInput(amountIn, tokenP, collat).amountOut > 0)</code>	✓	✓	
UT-SW-09	<code>mintExactOutputIsReachable</code>	Mint via <code>swapExactOutput</code> is reachable with positive input <code>satisfy(swapExactOutput(amountOut, collat, tokenP).amountIn > 0)</code>	✓	✓	
UT-SW-10	<code>burnExactOutputIsReachable</code>	Burn via <code>swapExactOutput</code> is reachable with positive input <code>satisfy(swapExactOutput(amountOut, tokenP, collat).amountIn > 0)</code>	✓	✓	

Parallelizer — Redeemer

Property	Name	Description	Audit	Mitig	Notes
UT-RD-01	<code>redeemBurnsExactAmount</code>	Sender's TokenP balance decreases by exactly amount <code>redeem(amount, ...) => bal[tokenP][sender] - bal'[tokenP][sender] = amount</code>	✓	✓	
UT-RD-02	<code>redeemDecreasesTokenPTotalSupply</code>	TokenP total supply decreases by exactly amount <code>redeem(amount, ...) => supply[tokenP] - supply'[tokenP] = amount</code>	✓	✓	
UT-RD-03	<code>redeemWithForfeitBurnsExactAmount</code>	Sender's balance decreases by exactly amount <code>redeemWithForfeit(amount, ...) => bal[tokenP][sender] - bal'[tokenP][sender] = amount</code>	✓	✓	
UT-RD-04	<code>redeemWithForfeitDecreasesTokenPTotalSupply</code>	Total supply decreases by exactly amount <code>redeemWithForfeit(amount, ...) => supply[tokenP] - supply'[tokenP] = amount</code>	✓	✓	
UT-RD-05	<code>updateNormalizerReturnMatchesGhost</code>	Return value matches post-call normalizer <code>retVal = updateNormalizer(amount, increase) => retVal = norm'</code>	✓	✓	
UT-RD-06	<code>redeemRevertsAfterDeadline</code>	Reverts when <code>block.timestamp > deadline</code> <code>block.timestamp > deadline => redeem(amount, ..., deadline, ...) REVERTS</code>	✓	✓	
UT-RD-07	<code>redeemRevertsWhenPaused</code>	Reverts when <code>isRedemptionLive == 0</code> <code>isRedemptionLive = 0 => redeem(amount, ...) REVERTS</code>	✓	✓	
UT-RD-08	<code>redeemWithForfeitRevertsAfterDeadline</code>	Reverts when <code>block.timestamp > deadline</code> <code>block.timestamp > deadline => redeemWithForfeit(amount, ..., deadline, ...) REVERTS</code>	✓	✓	
UT-RD-09	<code>redeemWithForfeitRevertsWhenPaused</code>	Reverts when <code>isRedemptionLive == 0</code> <code>isRedemptionLive = 0 => redeemWithForfeit(amount, ...) REVERTS</code>	✓	✓	
UT-RD-10	<code>updateNormalizerDoesNotChangeTokenPSupply</code>	Does not change TokenP total supply <code>updateNormalizer(amount, increase) => supply'[tokenP] = supply[tokenP]</code>	✓	✓	
UT-RD-11	<code>redeemIsolatesThirdPartyTokenPBalance</code>	Does not change third-party TokenP balances <code>third != sender => redeem(amount, ...) => bal'[tokenP][third] = bal[tokenP][third]</code>	✓	✓	
UT-RD-12	<code>redeemIsReachable</code>	Redeem is reachable with positive burn <code>satisfy(redeem(amount, ...) AND bal[tokenP][sender] - bal'[tokenP][sender] > 0)</code>	✓	✓	
UT-RD-13	<code>redeemWithForfeitIsReachable</code>	RedeemWithForfeit is reachable with positive burn <code>satisfy(redeemWithForfeit(amount, ...) AND bal[tokenP][sender] - bal'[tokenP][sender] > 0)</code>	✓	✓	
UT-RD-14	<code>updateNormalizerIsReachable</code>	UpdateNormalizer is reachable with normalizer change <code>satisfy(updateNormalizer(amount, increase) AND norm' != norm)</code>	✓	✓	
UT-RD-15	<code>quoteRedemptionCurveIsReachable</code>	QuoteRedemptionCurve is reachable with positive amount <code>satisfy(!REVERTS AND amount > 0)</code>	✓	✓	

Parallelizer — Setter

Property	Name	Description	Audit	Mitig	Notes
UT-SET-01a	<code>pauseMintIntegrity</code>	pauseMint sets <code>isMintLive</code> to 0 <code>pauseMint(t) => isMintLive'[t] == 0</code>	✓	✓	
UT-SET-01b	<code>unpauseMintIntegrity</code>	unpauseMint sets <code>isMintLive</code> to 1 <code>unpauseMint(t) => isMintLive'[t] == 1</code>	✓	✓	
UT-SET-02a	<code>pauseBurnIntegrity</code>	pauseBurn sets <code>isBurnLive</code> to 0 <code>pauseBurn(t) => isBurnLive'[t] == 0</code>	✓	✓	
UT-SET-02b	<code>unpauseBurnIntegrity</code>	unpauseBurn sets <code>isBurnLive</code> to 1 <code>unpauseBurn(t) => isBurnLive'[t] == 1</code>	✓	✓	

UT-SET-03a	<code>pauseRedeemIntegrity</code>	pauseRedeem sets isRedemptionLive to 0 <code>pauseRedeem() => isRedemptionLive' == 0</code>	✓	✓	
UT-SET-03b	<code>unpauseRedeemIntegrity</code>	unpauseRedeem sets isRedemptionLive to 1 <code>unpauseRedeem() => isRedemptionLive' == 1</code>	✓	✓	
UT-SET-04	<code>toggleTrustedUpdaterIntegrity</code>	toggleTrustedUpdater toggles ghostIsTrusted256 <code>toggleTrustedUpdater(a) => isTrusted'[a] == 1 - isTrusted[a]</code>	✓	✓	
UT-SET-05	<code>toggleTrustedSellerIntegrity</code>	toggleTrustedSeller toggles ghostIsSellerTrusted256 <code>toggleTrustedSeller(a) => isSellerTrusted'[a] == 1 - isSellerTrusted[a]</code>	✓	✓	
UT-SET-06	<code>updateSlippageToleranceIntegrity</code>	updateSlippageTolerance sets ghostSlippageTolerance256 to param <code>updateSlippageTolerance(t, val) => slippageTolerance'[t] == val</code>	✓	✓	
UT-SET-07	<code>updateSurplusBufferRatioIntegrity</code>	updateSurplusBufferRatio sets ghostSurplusBufferRatio64 to param <code>updateSurplusBufferRatio(val) => surplusBufferRatio' == val</code>	✓	✓	
UT-SET-08	<code>setStablecoinCapIntegrity</code>	setStablecoinCap sets ghostCollateralStablecoinCap256 to param <code>setStablecoinCap(t, cap) => stablecoinCap'[t] == cap</code>	✓	✓	
UT-SET-09	<code>setAccessManagerIntegrity</code>	setAccessManager sets ghostAccessManager to param <code>setAccessManager(addr) => accessManager' == addr</code>	✓	✓	
UT-SET-10a	<code>pauseMintRevertsNotCollateral</code>	pauseMint reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => pauseMint(t) REVERTS</code>	✓	✓	
UT-SET-10b	<code>unpauseMintRevertsNotCollateral</code>	unpauseMint reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => unpauseMint(t) REVERTS</code>	✓	✓	
UT-SET-11a	<code>pauseBurnRevertsNotCollateral</code>	pauseBurn reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => pauseBurn(t) REVERTS</code>	✓	✓	
UT-SET-11b	<code>unpauseBurnRevertsNotCollateral</code>	unpauseBurn reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => unpauseBurn(t) REVERTS</code>	✓	✓	
UT-SET-12	<code>adjustStablecoinsRevertsNotCollateral</code>	adjustStablecoins reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => adjustStablecoins(t, amt, inc) REVERTS</code>	✓	✓	
UT-SET-13	<code>updateSlippageToleranceRevertsInvalidRate</code>	updateSlippageTolerance reverts when slippageTolerance > BASE_9 <code>val > BASE_9 => updateSlippageTolerance(t, val) REVERTS</code>	✓	✓	
UT-SET-14	<code>updateSurplusBufferRatioRevertsInvalidParam</code>	updateSurplusBufferRatio reverts when ratio < BASE_9 <code>val < BASE_9 => updateSurplusBufferRatio(val) REVERTS</code>	✓	✓	
UT-SET-15	<code>setStablecoinCapRevertsNotCollateral</code>	setStablecoinCap reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => setStablecoinCap(t, cap) REVERTS</code>	✓	✓	
UT-SET-16	<code>revokeCollateralRevertsNotCollateral</code>	revokeCollateral reverts when collateral not registered (decimals == 0) <code>decimals[t] == 0 => revokeCollateral(t) REVERTS</code>	✓	✓	
UT-SET-17	<code>revokeCollateralRevertsCollateralBacked</code>	revokeCollateral reverts when collateral has backed stablecoins <code>decimals[t] != 0 AND collatNS[t] > 0 => revokeCollateral(t) REVERTS</code>	✓	✓	
UT-SET-18	<code>updateOracleRevertsNotTrusted</code>	updateOracle reverts when caller is not seller trusted <code>isSellerTrusted[msg.sender] == 0 => updateOracle(t) REVERTS</code>	✓	✓	
UT-SET-19a	<code>pauseMintDoesNotChangeBurnLive</code>	pauseMint does not change isBurnLive for same collateral <code>pauseMint(t) => isBurnLive'[t] == isBurnLive[t]</code>	✓	✓	

UT-SET-19b	<code>unpauseMintDoesNotChangeBurnLive</code>	unpauseMint does not change isBurnLive for same collateral <code>unpauseMint(t) => isBurnLive'[t] == isBurnLive[t]</code>	✓	✓	
UT-SET-20a	<code>pauseBurnDoesNotChangeMintLive</code>	pauseBurn does not change isMintLive for same collateral <code>pauseBurn(t) => isMintLive'[t] == isMintLive[t]</code>	✓	✓	
UT-SET-20b	<code>unpauseBurnDoesNotChangeMintLive</code>	unpauseBurn does not change isMintLive for same collateral <code>unpauseBurn(t) => isMintLive'[t] == isMintLive[t]</code>	✓	✓	
UT-SET-21a	<code>pauseMintDoesNotChangeRedemptionLive</code>	pauseMint does not change isRedemptionLive <code>pauseMint(t) => isRedemptionLive' == isRedemptionLive</code>	✓	✓	
UT-SET-21b	<code>unpauseMintDoesNotChangeRedemptionLive</code>	unpauseMint does not change isRedemptionLive <code>unpauseMint(t) => isRedemptionLive' == isRedemptionLive</code>	✓	✓	
UT-SET-22a	<code>pauseRedeemDoesNotChangeMintLive</code>	pauseRedeem does not change isMintLive for any collateral <code>forall t: pauseRedeem() => isMintLive'[t] == isMintLive[t]</code>	✓	✓	
UT-SET-22b	<code>unpauseRedeemDoesNotChangeMintLive</code>	unpauseRedeem does not change isMintLive for any collateral <code>forall t: unpauseRedeem() => isMintLive'[t] == isMintLive[t]</code>	✓	✓	
UT-SET-23	<code>toggleTrustedUpdaterDoesNotChangeSellerTrusted</code>	toggleTrustedUpdater does not change isSellerTrusted for same address <code>toggleTrustedUpdater(a) => isSellerTrusted'[a] == isSellerTrusted[a]</code>	✓	✓	
UT-SET-24	<code>toggleTrustedSellerDoesNotChangeTrusted</code>	toggleTrustedSeller does not change isTrusted for same address <code>toggleTrustedSeller(a) => isTrusted'[a] == isTrusted[a]</code>	✓	✓	
UT-SET-25	<code>adjustStablecoinsIsolatesOtherCollateral</code>	adjustStablecoins does not change other collateral's normalizedStables <code>t != other => adjustStablecoins(t, amt, inc) => collatNS'[other] == collatNS[other]</code>	✓	✓	
UT-SET-26	<code>updateSlippageToleranceIsolatesOtherCollateral</code>	updateSlippageTolerance does not change other collateral's tolerance <code>t != other => updateSlippageTolerance(t, val) => slippageTolerance'[other] == slippageTolerance[other]</code>	✓	✓	
UT-SET-27	<code>setStablecoinCapIsolatesOtherCollateral</code>	setStablecoinCap does not change other collateral's cap <code>t != other => setStablecoinCap(t, cap) => stablecoinCap'[other] == stablecoinCap[other]</code>	✓	✓	
UT-SET-28a	<code>pauseMintIsReachable</code>	pauseMint is reachable (sets isMintLive to 0) <code>satisfy(isMintLive'[t] == 0)</code>	✓	✓	
UT-SET-28b	<code>unpauseMintIsReachable</code>	unpauseMint is reachable (sets isMintLive to 1) <code>satisfy(isMintLive'[t] == 1)</code>	✓	✓	
UT-SET-29	<code>adjustStablecoinsIsReachable</code>	adjustStablecoins is reachable (global NS increases) <code>satisfy(NS' > NS)</code>	✓	✓	
UT-SET-30	<code>updateSlippageToleranceIsReachable</code>	updateSlippageTolerance is reachable (tolerance set to positive value) <code>satisfy(slippageTolerance'[t] > 0)</code>	✓	✓	
UT-SET-31	<code>updateSurplusBufferRatioIsReachable</code>	updateSurplusBufferRatio is reachable (ratio set to valid value >= BASE_9) <code>satisfy(surplusBufferRatio' >= BASE_9)</code>	✓	✓	
UT-SET-32	<code>setStablecoinCapIsReachable</code>	setStablecoinCap is reachable (cap set to positive value) <code>satisfy(stablecoinCap'[t] > 0)</code>	✓	✓	

Parallelizer — Surplus

Property	Name	Description	Audit	Mitig	Notes
UT-SUR-01	<code>releaseUpdatesLastReleasedAt</code>	lastReleasedAt == block.timestamp after release <code>lastReleasedAt' == block.timestamp</code>	✓	✓	
UT-SUR-02	<code>processSurplusRevertsWhenBufferRatioZero</code>	Reverts when surplusBufferRatio == 0 <code>surplusBufferRatio == 0 => REVERTS</code>	✓	✓	
UT-SUR-03	<code>releaseRevertsWhenIncomeZero</code>	Reverts when diamond TokenP balance == 0 <code>bal[TokenP][diamond] == 0 => REVERTS</code>	✓	✓	
UT-SUR-04	<code>releaseRevertsWhenNoPayees</code>	Reverts when payees array is empty <code>payeesLength == 0 => REVERTS</code>	✓	✓	
UT-SUR-05	<code>processSurplusDoesNotChangeLastReleasedAt</code>	Does not change lastReleasedAt <code>lastReleasedAt' == lastReleasedAt</code>	✓	✓	
UT-SUR-06	<code>processSurplusDoesNotChangeTotalShares</code>	Does not change totalShares <code>totalShares' == totalShares</code>	✓	✓	
UT-SUR-07	<code>processSurplusDoesNotChangePayeesLength</code>	Does not change payees.length <code>payeesLength' == payeesLength</code>	✓	✓	
UT-SUR-08	<code>processSurplusDoesNotChangeSurplusBufferRatio</code>	Does not change surplusBufferRatio <code>surplusBufferRatio' == surplusBufferRatio</code>	✓	✓	
UT-SUR-09	<code>releaseDoesNotChangeNormalizedStables</code>	Does not change global normalizedStables <code>NS' == NS</code>	✓	✓	
UT-SUR-10	<code>releaseDoesNotChangePerCollateralNs</code>	Does not change per-collateral normalizedStables <code>forall t: collatNS[t]' == collatNS[t]</code>	✓	✓	
UT-SUR-11	<code>releaseDoesNotChangeSurplusBufferRatio</code>	Does not change surplusBufferRatio <code>surplusBufferRatio' == surplusBufferRatio</code>	✓	✓	
UT-SUR-12	<code>releaseDoesNotChangeTotalShares</code>	Does not change totalShares <code>totalShares' == totalShares</code>	✓	✓	
UT-SUR-13	<code>processSurplusIsReachable</code>	Reachable with positive issuedAmount <code>satisfy(issuedAmount > 0)</code>	✓	✓	
UT-SUR-14	<code>processSurplusZeroIssuedIsReachable</code>	Reachable with zero issuedAmount <code>satisfy(issuedAmount == 0)</code>	✓	✓	
UT-SUR-15	<code>releaseIsReachable</code>	Release is reachable <code>satisfy(lastReleasedAt' == block.timestamp)</code>	✓	✓	

Savings

Property	Name	Description	Audit	Mitig	Notes
UT-SA-01	<code>depositIncreasesReceiverShares</code>	deposit increases receiver's share balance by returned shares <code>bal[shares][receiver]' == bal[shares][receiver] + deposit/assets).shares</code>	✓	✓	
UT-SA-02	<code>mintIncreasesReceiverShares</code>	mint increases receiver's share balance by requested shares <code>bal[shares][receiver]' == bal[shares][receiver] + shares</code>	✓	✓	
UT-SA-03	<code>depositTransfersAssetsFromSender</code>	deposit transfers exactly the deposited assets from sender <code>bal[asset][sender]' == bal[asset][sender] - assets</code>	✓	✓	
UT-SA-04	<code>withdrawDecreasesOwnerShares</code>	withdraw decreases owner's share balance by returned shares <code>bal[shares][owner]' == bal[shares][owner] - withdraw/assets).shares</code>	✓	✓	
UT-SA-05	<code>redeemTransfersAssetsToReceiver</code>	redeem transfers returned assets to receiver <code>bal[asset][receiver]' == bal[asset][receiver] + redeem(shares).assets</code>	✓	✓	

UT-SA-06	<code>depositIncreasesShareSupply</code>	Deposit increases share totalSupply by minted shares <code>supply' == supply + deposit(assets).shares</code>	✓	✓	
UT-SA-07	<code>redeemDecreasesShareSupply</code>	Redeem decreases share totalSupply by burned shares <code>supply' == supply - shares</code>	✓	✓	
UT-SA-08	<code>depositRefreshesLastUpdate</code>	Deposit sets lastUpdate to block.timestamp <code>lastUpdate' == block.timestamp after deposit</code>	✓	✓	
UT-SA-09	<code>mintRefreshesLastUpdate</code>	Mint sets lastUpdate to block.timestamp <code>lastUpdate' == block.timestamp after mint</code>	✓	✓	
UT-SA-10	<code>withdrawRefreshesLastUpdate</code>	Withdraw sets lastUpdate to block.timestamp <code>lastUpdate' == block.timestamp after withdraw</code>	✓	✓	
UT-SA-11	<code>redeemRefreshesLastUpdate</code>	Redeem sets lastUpdate to block.timestamp <code>lastUpdate' == block.timestamp after redeem</code>	✓	✓	
UT-SA-12	<code>depositIncreasesVaultAssets</code>	deposit does not decrease vault's TokenP balance <code>bal[asset][vault]' >= bal[asset][vault] after deposit</code>	✓	✓	
UT-SA-13a	<code>pauseSetsPausedFlag</code>	pause sets the paused flag to 1 <code>paused' == 1 after pause</code>	✓	✓	
UT-SA-13b	<code>unpauseClearsPausedFlag</code>	unpause clears the paused flag to 0 <code>paused' == 0 after unpause</code>	✓	✓	
UT-SA-14	<code>setRateRefreshesLastUpdate</code>	setRate accrues (sets lastUpdate) before changing rate <code>lastUpdate' == block.timestamp after setRate</code>	✓	✓	
UT-SA-15	<code>setRateStoresNewValue</code>	setRate stores the new rate value <code>rate' == newRate after setRate(newRate)</code>	✓	✓	
UT-SA-16	<code>setMaxRateStoresNewValue</code>	setMaxRate stores the new maxRate value <code>maxRate' == newMaxRate after setMaxRate(newMaxRate)</code>	✓	✓	
UT-SA-17	<code>toggleTrustedFlipsFlag</code>	toggleTrusted flips the trusted flag for the given address <code>isTrustedUpdater[addr]' == 1 - isTrustedUpdater[addr] after toggleTrusted(addr)</code>	✓	✓	
UT-SA-18	<code>depositZeroGivesZeroShares</code>	deposit(0) returns 0 shares <code>deposit(0).shares == 0</code>	✓	✓	
UT-SA-19	<code>redeemZeroGivesZeroAssets</code>	redeem(0) returns 0 assets <code>redeem(0).assets == 0</code>	✓	✓	
UT-SA-20	<code>mintZeroGivesZeroAssets</code>	mint(0) returns 0 assets <code>mint(0).assets == 0</code>	✓	✓	
UT-SA-21	<code>withdrawZeroGivesZeroShares</code>	withdraw(0) returns 0 shares <code>withdraw(0).shares == 0</code>	✓	✓	
UT-SA-22	<code>depositRevertsWhenPaused</code>	deposit reverts when paused <code>paused == 1 => deposit REVERTS</code>	✓	✓	
UT-SA-23	<code>mintRevertsWhenPaused</code>	mint reverts when paused <code>paused == 1 => mint REVERTS</code>	✓	✓	
UT-SA-24	<code>withdrawRevertsWhenPaused</code>	withdraw reverts when paused <code>paused == 1 => withdraw REVERTS</code>	✓	✓	
UT-SA-25	<code>redeemRevertsWhenPaused</code>	redeem reverts when paused <code>paused == 1 => redeem REVERTS</code>	✓	✓	
UT-SA-26	<code>setRateRejectsRateAboveMax</code>	setRate reverts when newRate exceeds maxRate <code>newRate > maxRate => setRate(newRate) REVERTS</code>	✓	✓	
UT-SA-27	<code>depositDoesNotChangeRate</code>	deposit writes lastUpdate but not rate <code>rate' == rate after deposit</code>	✓	✓	
UT-SA-28	<code>depositDoesNotChangePaused</code>	deposit writes lastUpdate but not paused <code>paused' == paused after deposit</code>	✓	✓	
UT-SA-29	<code>setRateDoesNotChangePaused</code>	setRate writes rate but not paused <code>paused' == paused after setRate</code>	✓	✓	

UT-SA-30a	<code>pauseDoesNotChangeRate</code>	pause writes paused but not rate <code>rate' == rate after pause</code>	✓	✓	
UT-SA-30b	<code>unpauseDoesNotChangeRate</code>	unpause writes paused but not rate <code>rate' == rate after unpause</code>	✓	✓	
UT-SA-31a	<code>pauseRefreshesLastUpdate</code>	pause settles outstanding yield via <code>_accrue</code> , advancing <code>lastUpdate</code> to now <code>lastUpdate' == block.timestamp after pause</code>	—	✓	pause settles outstanding yield via <code>_accrue</code> (post-fix behaviour)
UT-SA-31b	<code>unpauseRefreshesLastUpdate</code>	unpause settles outstanding yield via <code>_accrue</code> , advancing <code>lastUpdate</code> to now <code>lastUpdate' == block.timestamp after unpause</code>	—	✓	unpause settles outstanding yield via <code>_accrue</code> (post-fix behaviour)
UT-SA-32a	<code>pauseDoesNotChangeTotalSupply</code>	pause must not affect share <code>totalSupply</code> <code>supply' == supply after pause</code>	✓	✓	
UT-SA-32b	<code>unpauseDoesNotChangeTotalSupply</code>	unpause must not affect share <code>totalSupply</code> <code>supply' == supply after unpause</code>	✓	✓	
UT-SA-33	<code>setRateDoesNotChangeTotalSupply</code>	<code>setRate</code> accrues <code>TokenP</code> but must not change share <code>totalSupply</code> <code>supply' == supply after setRate</code>	✓	✓	
UT-SA-34	<code>setMaxRateClampsRateToCap</code>	<code>setMaxRate</code> clamps the active rate down to the new cap, else leaves it unchanged <code>rate' == min(rate, newMaxRate) after setMaxRate</code>	—	✓	<code>setMaxRate</code> now clamps rate down to the new cap (post-fix behaviour)
UT-SA-35	<code>depositDoesNotAffectThirdPartyShares</code>	Deposit does not change third party's share balance <code>third != receiver => bal[shares][third]' == bal[shares][third] after deposit</code>	✓	✓	
UT-SA-36	<code>redeemDoesNotAffectThirdPartyShares</code>	Redeem does not change third party's share balance <code>third != owner => bal[shares][third]' == bal[shares][third] after redeem</code>	✓	✓	
UT-SA-37	<code>depositDoesNotAffectThirdPartyAssets</code>	Deposit does not change third party's <code>TokenP</code> balance <code>third != sender AND third != vault => bal[asset][third]' == bal[asset][third] after deposit</code>	✓	✓	
UT-SA-38	<code>withdrawDoesNotAffectThirdPartyAssets</code>	Withdraw does not change third party's <code>TokenP</code> balance <code>third != receiver AND third != vault => bal[asset][third]' == bal[asset][third] after withdraw</code>	✓	✓	
UT-SA-39	<code>toggleTrustedDoesNotAffectOtherAddresses</code>	<code>toggleTrusted</code> does not change other addresses <code>other != addr => isTrustedUpdater[other]' == isTrustedUpdater[other] after toggleTrusted(addr)</code>	✓	✓	
UT-SA-40	<code>estimatedAPYSameForAnyCaller</code>	<code>estimatedAPY</code> returns the same result regardless of caller <code>sender1 != sender2 => estimatedAPY(e1) == estimatedAPY(e2)</code>	✓	✓	
UT-SA-41	<code>totalAssetsSameForAnyCaller</code>	<code>totalAssets</code> returns the same result regardless of caller <code>sender1 != sender2 => totalAssets(e1) == totalAssets(e2)</code>	✓	✓	
UT-SA-42	<code>estimatedAPYNeverReverts</code>	<code>estimatedAPY</code> never reverts under valid state <code>!REVERTS(estimatedAPY())</code>	✓	✓	
UT-SA-43	<code>depositIsReachable</code>	Deposit can succeed with positive shares <code>satisfy(deposit(assets).shares > 0)</code>	✓	✓	
UT-SA-44	<code>withdrawIsReachable</code>	Withdraw can succeed with positive shares burned <code>satisfy(withdraw(assets).shares > 0)</code>	✓	✓	
UT-SA-45	<code>mintIsReachable</code>	Mint can succeed with positive assets paid <code>satisfy(mint(shares).assets > 0)</code>	✓	✓	
UT-SA-46	<code>redeemIsReachable</code>	Redeem can succeed with positive assets returned <code>satisfy(redeem(shares).assets > 0)</code>	✓	✓	

FlashParallelToken

Property	Name	Description	Audit	Mitig	Notes
UT-FPT-01	<code>setFeeRecipientStoresNewValue</code>	Correctly stores the new recipient <code>feeRecipient' == newRecipient</code> after <code>setFlashLoanFeeRecipient(newRecipient)</code>	✓	✓	
UT-FPT-02	<code>setParametersStoresAllFields</code>	Correctly stores fee rate, max borrowable, and active flag <code>feesRate[t]' == feesRate</code> AND <code>maxBorrowable[t]' == maxBorrowable</code> AND <code>isActive[t]' == isActive</code> after <code>setFlashLoanParameters(t, feesRate, maxBorrowable, isActive)</code>	✓	✓	
UT-FPT-03	<code>toggleFlipsActiveFlag</code>	Flips the isActive flag <code>isActive[t]' != isActive[t]</code> after <code>toggleActiveToken(t)</code>	✓	✓	
UT-FPT-04	<code>maxFlashLoanReturnsCorrectLimit</code>	Returns maxBorrowable when active, zero when inactive <code>maxFlashLoan(t) == (isActive[t] ? maxBorrowable[t] : 0)</code>	✓	✓	
UT-FPT-05	<code>flashFeeReturnsCorrectAmount</code>	Returns amount * feeRate with half-up rounding <code>flashFee(t, amount) == (amount * feesRate[t] + 5000) / 10000</code>	✓	✓	
UT-FPT-06	<code>setFeeRecipientRejectsZeroAddress</code>	Reverts on zero address <code>newRecipient == 0 => REVERTS</code> after <code>setFlashLoanFeeRecipient(newRecipient)</code>	✓	✓	
UT-FPT-07	<code>setParametersRejectsFeeRateAboveMax</code>	Reverts when fee rate exceeds 100% <code>feesRate > PERCENTAGE_FACTOR => REVERTS</code> after <code>setFlashLoanParameters(t, feesRate, ...)</code>	✓	✓	
UT-FPT-08	<code>flashFeeRejectsInactiveToken</code>	Reverts for inactive tokens <code>!isActive[t] => REVERTS</code> after <code>flashFee(t, amount)</code>	✓	✓	
UT-FPT-09	<code>flashLoanRejectsInactiveToken</code>	Reverts for inactive tokens <code>!isActive[t] => REVERTS</code> after <code>flashLoan(receiver, t, amount, data)</code>	✓	✓	
UT-FPT-10	<code>flashLoanRejectsAmountAboveMax</code>	Reverts when amount exceeds max borrowable <code>amount > maxBorrowable[t] => REVERTS</code> after <code>flashLoan(receiver, t, amount, data)</code>	✓	✓	
UT-FPT-11	<code>setFeeRecipientDoesNotChangeTokenSettings</code>	Does not change any token settings <code>config[t]' == config[t]</code> after <code>setFlashLoanFeeRecipient(newRecipient)</code>	✓	✓	
UT-FPT-12	<code>setParametersDoesNotChangeFeeRecipient</code>	Does not change the fee recipient <code>feeRecipient' == feeRecipient</code> after <code>setFlashLoanParameters(t, feesRate, maxBorrowable, isActive)</code>	✓	✓	
UT-FPT-13	<code>setParametersDoesNotAffectOtherTokens</code>	Does not affect other tokens <code>forall o != t: config[o]' == config[o]</code> after <code>setFlashLoanParameters(t, ...)</code>	✓	✓	
UT-FPT-14	<code>toggleDoesNotChangeFeeRecipient</code>	Does not change the fee recipient <code>feeRecipient' == feeRecipient</code> after <code>toggleActiveToken(t)</code>	✓	✓	
UT-FPT-15	<code>toggleDoesNotChangeFeeRateOrMaxBorrowable</code>	Does not change fee rate or max borrowable <code>feesRate[t]' == feesRate[t]</code> AND <code>maxBorrowable[t]' == maxBorrowable[t]</code> after <code>toggleActiveToken(t)</code>	✓	✓	
UT-FPT-16	<code>toggleDoesNotAffectOtherTokens</code>	Does not affect other tokens <code>forall o != t: config[o]' == config[o]</code> after <code>toggleActiveToken(t)</code>	✓	✓	
UT-FPT-17	<code>flashLoanDoesNotChangeAnyConfig</code>	Does not modify fee recipient or token settings <code>feeRecipient' == feeRecipient</code> AND <code>config[t]' == config[t]</code> after <code>flashLoan(receiver, t, amount, data)</code>	✓	✓	
UT-FPT-18	<code>maxFlashLoanSameForAnyCaller</code>	Returns same result regardless of caller <code>forall e1, e2: maxFlashLoan(e1, t) == maxFlashLoan(e2, t)</code> at same state	✓	✓	
UT-FPT-19	<code>flashFeeSameForAnyCaller</code>	Returns same result regardless of caller <code>forall e1, e2: flashFee(e1, t, amount) == flashFee(e2, t, amount)</code> at same state	✓	✓	
UT-FPT-20	<code>maxFlashLoanNeverReverts</code>	Never reverts <code>!REVERTS</code> after <code>maxFlashLoan(t)</code>	✓	✓	
UT-FPT-21	<code>setFeeRecipientIsReachable</code>	Can succeed <code>satisfy(feeRecipient' == newRecipient)</code>	✓	✓	
UT-FPT-	<code>setParametersIsReachable</code>	Can succeed with non-trivial params	✓	✓	

22		<code>satisfy(feesRate > 0 AND maxBorrowable > 0 AND isActive)</code>			
UT-FPT-23	<code>toggleActiveTokenIsReachable</code>	Can succeed and activate a token <code>satisfy(isActive[t] == true)</code>	✓	✓	
UT-FPT-24	<code>flashLoanIsReachable</code>	Can succeed with non-zero borrow <code>satisfy(amount > 0) after flashLoan(receiver, t, amount, data)</code>	✓	✓	
UT-FPT-25	<code>flashFeeIsReachable</code>	Can return a non-zero fee <code>satisfy(flashFee(t, amount) > 0)</code>	✓	✓	
UT-FPT-26	<code>accrueInterestTransfersExactBalance</code>	Fee recipient receives exactly the contract's balance <code>bal[t][contract]' == 0 AND bal[t][feeRecipient]' == bal[t][feeRecipient] + bal[t][contract] after accrueInterest(t)</code>	✓	✓	
UT-FPT-27	<code>accrueInterestDoesNotChangeConfig</code>	Does not change any config <code>feeRecipient' == feeRecipient AND config[t]' == config[t] after accrueInterest(token)</code>	✓	✓	
UT-FPT-28	<code>accrueInterestDoesNotChangeTotalSupply</code>	Does not change total supply <code>supply[t]' == supply[t] after accrueInterest(t)</code>	✓	✓	
UT-FPT-29	<code>accrueInterestIsReachable</code>	Can succeed with non-zero balance <code>satisfy(bal[t][contract] > 0) after accrueInterest(t)</code>	✓	✓	

ERC-4626 Conformance

Properties verifying conformance to the ERC-4626 tokenized vault standard (Savings).

Property	Name	Description	Audit	Mitig	Notes
ERC4626-01	<code>assetReturnsConfiguredToken</code>	asset() returns the configured ERC-20 token <code>asset() == configuredAsset</code>	✓	✓	
ERC4626-02	<code>assetNeverReverts</code>	asset() never reverts <code>!REVERTS(asset())</code>	✓	✓	
ERC4626-03	<code>totalAssetsIncludesAccruedInterest</code>	totalAssets is the projection of the tracked accrual base, never below it <code>totalAssets() >= storedAssets</code>	✓	✓	Post-fix the accrual base is <code>storedAssets</code> , not the raw ERC-20 balance (donations are intentionally excluded) — see Real Issue #5
ERC4626-04	<code>totalAssetsNeverReverts</code>	totalAssets() never reverts <code>!REVERTS(totalAssets())</code>	✓	✓	
ERC4626-05	<code>convertToSharesSameForAnyCaller</code>	Caller-agnostic <code>convertToShares(e1, assets) == convertToShares(e2, assets) for any e1, e2</code>	✓	✓	
ERC4626-06	<code>convertToSharesNeverReverts</code>	Never reverts with reasonable input <code>assets < max_uint96 AND supply > 0 => !REVERTS(convertToShares(assets))</code>	✓	✓	
ERC4626-07	<code>convertToSharesRoundsDown</code>	Rounds toward zero <code>convertToAssets(convertToShares(assets)) <= assets</code>	✓	✓	
ERC4626-08	<code>convertToAssetsSameForAnyCaller</code>	Caller-agnostic <code>convertToAssets(e1, shares) == convertToAssets(e2, shares) for any e1, e2</code>	✓	✓	
ERC4626-09	<code>convertToAssetsNeverReverts</code>	Never reverts with reasonable input <code>shares < max_uint96 => !REVERTS(convertToAssets(shares))</code>	✓	✓	
ERC4626-10	<code>convertToAssetsRoundsDown</code>	Rounds toward zero <code>convertToShares(convertToAssets(shares)) <= shares</code>	✓	✓	
ERC4626-11	<code>maxDepositNeverReverts</code>	maxDeposit() never reverts <code>!REVERTS(maxDeposit(receiver))</code>	✓	✓	
ERC4626-12	<code>maxMintNeverReverts</code>	maxMint() never reverts <code>!REVERTS(maxMint(receiver))</code>	✓	✓	
ERC4626-13	<code>depositAboveMaxReverts</code>	Deposit above maxDeposit limit reverts <code>maxDeposit != max_uint256 AND assets > maxDeposit => deposit REVERTS</code>	✓	✓	
ERC4626-14	<code>mintAboveMaxReverts</code>	Mint above maxMint limit reverts <code>maxMint != max_uint256 AND shares > maxMint => mint REVERTS</code>	✓	✓	
ERC4626-		Zero maxMint disables minting			

15	<code>zeroMaxMintBlocksMinting</code>	<code>maxMint == 0 AND shares > 0 => mint REVERTS</code>	✓	✓	
ERC4626-16	<code>withdrawAboveMaxReverts</code>	Withdraw above maxWithdraw limit reverts <code>assets > maxWithdraw(owner) => withdraw REVERTS</code>	✓	✓	
ERC4626-17	<code>maxWithdrawNeverReverts</code>	maxWithdraw() never reverts <code>!REVERTS(maxWithdraw(owner))</code>	✓	✓	
ERC4626-18	<code>maxRedeemNeverReverts</code>	maxRedeem() never reverts <code>!REVERTS(maxRedeem(owner))</code>	✓	✓	
ERC4626-19	<code>redeemAboveMaxReverts</code>	Redeem above maxRedeem limit reverts <code>shares > maxRedeem(owner) => redeem REVERTS</code>	✓	✓	
ERC4626-20	<code>zeroMaxRedeemBlocksRedemption</code>	Zero maxRedeem disables redemption <code>maxRedeem == 0 AND shares > 0 => redeem REVERTS</code>	✓	✓	
ERC4626-21	<code>previewDepositNeverOverestimates</code>	Actual shares ≥ previewed shares <code>deposit/assets).shares >= previewDeposit/assets)</code>	✓	✓	
ERC4626-22	<code>previewMintNeverUnderestimates</code>	Actual assets ≤ previewed assets <code>mint(shares).assets <= previewMint(shares)</code>	✓	✓	
ERC4626-23	<code>previewWithdrawNeverUnderestimates</code>	Actual shares ≤ previewed shares <code>withdraw/assets).shares <= previewWithdraw/assets)</code>	✓	✓	
ERC4626-24	<code>previewRedeemNeverOverestimates</code>	Actual assets ≥ previewed assets <code>redeem(shares).assets >= previewRedeem(shares)</code>	✓	✓	
ERC4626-25	<code>previewDepositIncludesFees</code>	Fee-inclusive (shares ≤ ideal) <code>previewDeposit/assets) <= convertToShares/assets)</code>	✓	✓	
ERC4626-26	<code>previewMintIncludesFees</code>	Fee-inclusive (assets ≥ ideal) <code>previewMint(shares) >= convertToAssets(shares)</code>	✓	✓	
ERC4626-27	<code>previewWithdrawIncludesFees</code>	Fee-inclusive (shares ≥ ideal) <code>previewWithdraw/assets) >= convertToShares/assets)</code>	✓	✓	
ERC4626-28	<code>previewRedeemIncludesFees</code>	Fee-inclusive (assets ≤ ideal) <code>previewRedeem(shares) <= convertToAssets(shares)</code>	✓	✓	
ERC4626-29	<code>previewDepositSameForAnyCaller</code>	Caller-agnostic <code>previewDeposit(e1, assets) == previewDeposit(e2, assets)</code> for any e1, e2	✓	✓	
ERC4626-30	<code>previewMintSameForAnyCaller</code>	Caller-agnostic <code>previewMint(e1, shares) == previewMint(e2, shares)</code> for any e1, e2	✓	✓	
ERC4626-31	<code>previewWithdrawSameForAnyCaller</code>	Caller-agnostic <code>previewWithdraw(e1, assets) == previewWithdraw(e2, assets)</code> for any e1, e2	✓	✓	
ERC4626-32	<code>previewRedeemSameForAnyCaller</code>	Caller-agnostic <code>previewRedeem(e1, shares) == previewRedeem(e2, shares)</code> for any e1, e2	✓	✓	
ERC4626-33	<code>previewDepositRevertsOnlyIfDepositReverts</code>	Revert correlation <code>REVERTS(previewDeposit/assets)) => REVERTS(deposit/assets))</code>	✓	✓	
ERC4626-34	<code>previewMintRevertsOnlyIfMintReverts</code>	Revert correlation <code>REVERTS(previewMint(shares)) => REVERTS(mint(shares))</code>	✓	✓	
ERC4626-35	<code>previewWithdrawRevertsOnlyIfWithdrawReverts</code>	Revert correlation <code>REVERTS(previewWithdraw/assets)) => REVERTS(withdraw/assets))</code>	✓	✓	
ERC4626-36	<code>previewRedeemRevertsOnlyIfRedeemReverts</code>	Revert correlation <code>REVERTS(previewRedeem(shares)) => REVERTS(redeem(shares))</code>	✓	✓	
ERC4626-37	<code>depositTransfersAndMintsCorrectly</code>	Transfers exact assets, mints correct shares <code>bal[asset][sender]' == bal[asset][sender] - assets AND bal[shares][receiver]' == bal[shares][receiver] + shares AND supply' == supply + shares</code>	✓	✓	
ERC4626-38	<code>depositRevertsIfAssetsNotFullyTransferred</code>	Reverts if assets not fully transferred <code>bal[asset][sender]' != bal[asset][sender] - assets => deposit REVERTS</code>	✓	✓	
ERC4626-39	<code>depositRespectsAssetAllowance</code>	Respects ERC20 allowance <code>allowance[asset][sender][vault] != max_uint256 AND allowance < assets => deposit REVERTS</code>	✓	✓	

ERC4626-40	<code>mintTransfersAndMintsCorrectly</code>	Transfers computed assets, mints exact shares <code>bal[asset][sender]' == bal[asset][sender] - assets AND bal[shares][receiver]' == bal[shares][receiver] + shares AND supply' == supply + shares</code>	✓	✓	
ERC4626-41	<code>mintRevertsIfSharesNotFullyMinted</code>	Reverts if shares not fully minted <code>bal[shares][receiver]' != bal[shares][receiver] + shares => mint REVERTS</code>	✓	✓	
ERC4626-42	<code>mintDecreasesAllowanceByExactCost</code>	Decreases allowance by exact assets spent <code>allowance[asset][sender][vault] - allowance[asset][sender][vault]' == mint(shares).assets</code>	✓	✓	
ERC4626-43	<code>withdrawBurnsAndTransfersCorrectly</code>	Burns correct shares, transfers exact assets <code>bal[shares][owner]' == bal[shares][owner] - sharesBurned AND bal[asset][receiver]' == bal[asset][receiver] + assets AND supply' == supply - sharesBurned</code>	✓	✓	
ERC4626-44	<code>withdrawRevertsIfAssetsNotFullyReceived</code>	Reverts if assets not fully received <code>receiver != vault AND bal[asset][receiver]' != bal[asset][receiver] + assets => withdraw REVERTS</code>	✓	✓	
ERC4626-45	<code>redeemBurnsAndTransfersCorrectly</code>	Burns exact shares, transfers computed assets <code>bal[shares][owner]' == bal[shares][owner] - shares AND bal[asset][receiver]' == bal[asset][receiver] + assetsOut AND supply' == supply - shares</code>	✓	✓	
ERC4626-46	<code>redeemRevertsIfSharesNotFullyBurned</code>	Reverts if shares not fully burned <code>bal[shares][owner]' != bal[shares][owner] - shares => redeem REVERTS</code>	✓	✓	
ERC4626-47	<code>withdrawRespectsShareAllowance</code>	Respects share allowance <code>sender != owner AND allowance[shares][owner][sender] != max_uint256 AND allowance < sharesBurned => withdraw REVERTS</code>	✓	✓	
ERC4626-48	<code>redeemRespectsShareAllowance</code>	Respects share allowance <code>sender != owner AND allowance[shares][owner][sender] != max_uint256 AND allowance < shares => redeem REVERTS</code>	✓	✓	
ERC4626-49	<code>withdrawDecreasesAllowanceByBurnedShares</code>	Decreases allowance by burned shares <code>allowance[shares][owner][sender] - allowance[shares][owner][sender]' == withdraw(assets).sharesBurned</code>	✓	✓	
ERC4626-50	<code>redeemDecreasesAllowanceByRedeemedShares</code>	Decreases allowance by redeemed shares <code>allowance[shares][owner][sender] - allowance[shares][owner][sender]' == shares</code>	✓	✓	
ERC4626-51	<code>donationProfitBoundedByShareRatio</code>	Atomic donate-then-redeem must not yield payout above attacker's proportional share <code>previewRedeem(s)' - previewRedeem(s) <= donation * s / totalSupply</code>	✗	✓	At audit: dormant-accrual donation attack — see Real Issue #5 ; verifies after the fix

Real Issues Found

Five real bugs were identified during the security engagement. For each of them, the formal verification suite confirms the fix: rules that are violated on the audit commit verify on the mitigated code.

1. [HIGH] `Surplus::processSurplus` always reverts for managed collateral [#5](#)

`LibSurplus::_computeCollateralSurplus` reads the collateral balance from `LibManager::totalAssets` for managed collateral, which returns the balance held by the external strategy. It then computes a `collateralSurplus`. However, `Surplus::processSurplus` self-calls `swapExactInput(collateralSurplus, ...)`, and since this is an external self-call, `msg.sender` is the diamond — which holds zero balance for managed collateral. The transfer always reverts.

This is a permanent DoS on surplus processing for all managed collateral. Strategy yield cannot be captured as distributable TokenP.

❌ Violated:

```
// HL-S-10: processSurplus reachable for managed collateral
// FORMULA: satisfy(isManaged[collateral] > 0 AND collateralSurplus > 0 AND issuedAmount > 0)
rule processSurplusManagedCollateralIsReachable(
    env e,
    address collateral,
    uint256 maxCollateralAmount
) {
    setupValidStateParallelizer(e);

    require(ghostCollateralIsManaged8[collateral] > 0,
        "SAFE: test managed collateral path");

    uint256 collateralSurplus;
    uint256 stableSurplus;
    uint256 issuedAmount;
    collateralSurplus, stableSurplus, issuedAmount = processSurplus(e, collateral, maxCollateralAmount);

    satisfy(collateralSurplus > 0 && issuedAmount > 0,
        "processSurplus must be reachable for managed collateral");
}
```

✅ Passed (after the fix):

Before the self-swap, withdraw the surplus from the strategy:

```
+if (collatInfo.isManaged > 0) {
+    LibManager.release(collateral, address(this), collateralSurplus, collatInfo.managerData.config);
+}
```

2. [MEDIUM] `LibSurplus::_computeCollateralSurplus` doesn't account for `surplusBufferRatio > 100%` [#29](#)

`_computeCollateralSurplus` treats as surplus everything above 100% collateral ratio, but then uses `surplusBufferRatio` for the final check. When `surplusBufferRatio > 100%`, `processSurplus()` always reverts because the surplus computed at 100% CR is less than what's needed to maintain the buffer ratio. The `surplusBufferRatio` parameter effectively has no meaningful purpose.

❌ Violated:

```
// HL-S-09: processSurplus reachable when surplusBufferRatio > 100%
// FORMULA: satisfy(surplusBufferRatio > BASE_9 AND collateralSurplus > 0 AND issuedAmount > 0)
rule processSurplusReachableWithHighBufferRatio(
    env e,
    address collateral,
    uint256 maxCollateralAmount
) {
```

```

    setupValidStateParallelizer(e);

    require(ghostSurplusBufferRatio64 > BASE_9(),
        "SAFE: buffer ratio > 100%");

    uint256 collateralSurplus;
    uint256 stableSurplus;
    uint256 issuedAmount;
    collateralSurplus, stableSurplus, issuedAmount = processSurplus(e, collateral, maxCollateralAmount);

    satisfy(collateralSurplus > 0 && issuedAmount > 0,
        "processSurplus reachable with surplusBufferRatio > 100%");
}

```

✅ Passed (after the fix):

Use `surplusBufferRatio` to compute the maximum backable amount before checking for surplus:

```

-if (totalCollateralValue <= stablesBacked) revert ZeroSurplusAmount();
-stableSurplus = totalCollateralValue - stablesBacked;
+uint256 maxBackable = (totalCollateralValue * BASE_9) / ts.surplusBufferRatio;
+if (maxBackable <= stablesBacked) revert ZeroSurplusAmount();
+stableSurplus = maxBackable - stablesBacked;

```

3. [LOW] `SettersGovernor::setWhitelistStatus` allows values other than 0 and 1 [#16](#)

The `toggleWhitelist` function ensures the `whitelistStatus` field stays boolean via `1 - current`. But `setWhitelistStatus` accepts arbitrary `uint8` values (2–255). If a governor passes any of these, it causes a DoS on any function that calls `LibWhitelist::checkWhitelist`.

❌ Violated:

```

// VS-P-14: collateral onlyWhitelisted is boolean
// FORMULA: forall t: onlyWhitelisted[t] in {0, 1}
invariant collateralOnlyWhitelistedFlag(env e)
    forall address token.
        ghostCollateralOnlyWhitelisted8[token] == 0 || ghostCollateralOnlyWhitelisted8[token] == 1
filtered { f -> !EXCLUDED_FUNCTION(f) } { preserved with (env eFunc) { SETUP_P(e, eFunc); } }

```

✅ Passed (after the fix):

Reject non-boolean values at the entry point:

```

function setWhitelistStatus(address collateral, uint8 whitelistStatus, bytes memory whitelistData) internal {
+   if (whitelistStatus > 1) revert InvalidWhitelistStatus();
    Collateral storage collatInfo = s.transmuterStorage().collaterals[collateral];
}

```

4. [LOW] `collatInfo.stablecoinCap` hardcap can be bypassed via `SettersGovernor::adjustStablecoins` [#18](#)

The `stablecoinCap` is correctly enforced during normal user mint operations in the Swapper facet. However, `adjustStablecoins` can arbitrarily increase `normalizedStables` without checking against `stablecoinCap`, allowing the system to enter a state where a collateral backs more stablecoins than its configured limit.

❌ Violated:

```

// ST-P-08: Per-collateral normalizedStables increase respects stablecoinCap
// FORMULA: forall f, t: collatNS[t]' > collatNS[t] AND cap[t] > 0
//          => collatNS[t]' * norm' / BASE_27 <= cap[t]
rule stablecoinCapEnforcedOnNsIncrease(env e, method f, calldataarg args, address token)

```



```
filtered { f -> !EXCLUDED_FUNCTION(f) } {

  setupValidStateParallelizer(e);

  mathhint collatNsBefore = ghostCollateralNormalizedStables216[token];
  mathhint cap = ghostCollateralStablecoinCap256[token];

  f(e, args);

  mathhint collatNsAfter = ghostCollateralNormalizedStables216[token];
  mathhint normalizerAfter = ghostNormalizer128;

  assert(collatNsAfter > collatNsBefore && cap > 0
    => collatNsAfter * normalizerAfter / BASE_27() <= cap,
    "Per-collateral normalizedStables increase must respect stablecoinCap");
}
```

✔ Passed (after the fix):

Enforce the cap inside `adjustStablecoins` when increasing:

```
function adjustStablecoins(address collateral, uint128 amount, bool increase) internal {
  ...
  if (increase) {
    collatInfo.normalizedStables = collatInfo.normalizedStables + uint216(normalizedAmount);
    ts.normalizedStables = ts.normalizedStables + normalizedAmount;
+   if (collatInfo.normalizedStables * ts.normalizer / BASE_27 > collatInfo.stablecoinCap) {
+     revert AboveCap();
+   }
  } else {...}
}
```

The CVL rule quoted above is the audit-commit version (it triggered on raw `collatNS`). During the mitigation review the trigger was refined to the implied issued value `collatNS * normalizer` — see the ST-P-08 notes in [State Transitions](#).

5. `savings` dormant-accrual donation attack — `totalAssets` derived from raw ERC-20 balance ([PR #26](#))

Pre-fix, the Savings vault derived its accrual base from `IERC20(asset()).balanceOf(address(this))`. A direct ERC-20 transfer into the vault (a "donation") therefore fed straight into the rate-multiplied accrual computation: `totalAssets()` reported `_computeUpdatedAssets(balance + donation, dt)`, amplifying the donation by the accrued-interest factor. An attacker holding shares could donate, then immediately redeem at an inflated share price, extracting more than their proportional capture of the donation — and the inflated `totalAssets` overstated backing that the vault did not hold.

Three implementation-agnostic safety properties capture the vulnerability. Each is **violated on the pre-fix code** (concrete counter-example) and **verified on the post-fix code** — public Certora cloud runs (v8.13.0, no login required):

Property	Pre-fix (vulnerable)	Post-fix (fixed)
HL-SA-16 <code>donationDoesNotInflateShareValue</code>	✗ run	✔ run
HL-SA-17 <code>donationDoesNotAmplifyTotalAssets</code>	✗ run	✔ run
ERC4626-51 <code>donationProfitBoundedByShareRatio</code>	✗ run	✔ run

Dashboard counts: each job runs the property plus the auxiliary `envfreeFuncsStaticCheck`, so `2/2 Verified` means the property holds, `1/2` means a counter-example was found.

✗ Violated (the strongest of the three — atomic donate-then-redeem must not pay out more than the attacker's proportional share of the donation):

```
// ERC4626-51: donation profit bounded by share ratio
// FORMULA: previewRedeem(s) after donation D - previewRedeem(s) before <= D * s / totalSupply
rule donationProfitBoundedByShareRatio(env e, address attacker, uint256 donation) { ... }
```

✅ Passed (after the fix):

Track principal internally in `storedAssets` instead of deriving it from the raw ERC-20 balance, so donations never enter the accrual base (donated surplus stays out of `totalAssets()` and is recoverable via `recoverSurplus`):

```
+ /// @notice Tracked balance of `asset` backing share holders
+ uint256 public storedAssets;

function _accrue() internal returns (uint256 newTotalAssets) {
-   uint256 currentBalance = IERC20Metadata(asset()).balanceOf(address(this));
+   uint256 currentBalance = storedAssets;
    newTotalAssets = _computeUpdatedAssets(currentBalance, block.timestamp - lastUpdate);
    ...
}

function totalAssets() public view override returns (uint256) {
-   return _computeUpdatedAssets(IERC20Metadata(asset()).balanceOf(address(this)), block.timestamp - lastUpdate);
+   return _computeUpdatedAssets(storedAssets, block.timestamp - lastUpdate);
}
```

Mutation Testing

Valid state invariants are powerful — a single well-crafted invariant can catch a wide range of bugs across multiple functions. To demonstrate this, we use `normalizedStablesSumConservation` (VS-P-15) as a case study, showing through mutation testing that this one invariant detects 18 distinct bugs across `Swapper._swap()`, `LibSetters.adjustStablecoins()`, and `Redeemer._updateNormalizer()`.

What is Mutation Testing

Mutation testing validates the strength of a formal specification by injecting small syntactic changes (mutations) into the source code and verifying that the spec catches them. Each mutation represents a potential bug — a removed assignment, a flipped operator, an off-by-one error.

The [certoraMutate](#) tool automates this process: it applies each mutant to the source, runs the prover against the specified rule, and reports whether the invariant caught the introduced defect.

VS-P-15: normalizedStablesSumConservation

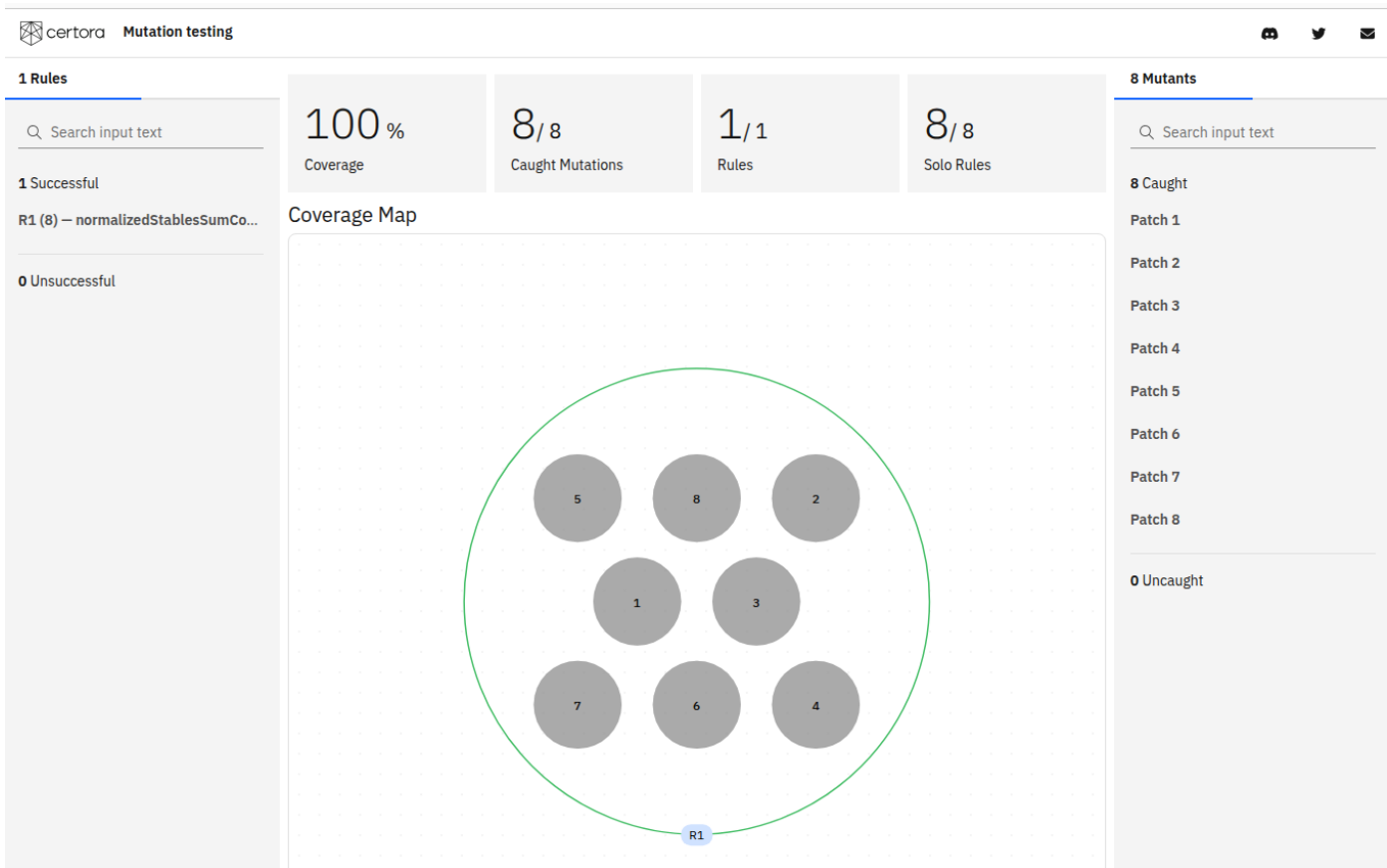
The `normalizedStablesSumConservation` invariant asserts that the global `normalizedStables` always equals the sum of per-collateral `normalizedStables` values. This strict equality makes the invariant maximally sensitive to any accounting discrepancy in the protocol's stablecoin bookkeeping.

```
// VS-P-15: global normalizedStables == sum of per-collateral normalizedStables
// FORMULA: NS == sum(collatNS[t] for t in collateralList)
invariant normalizedStablesSumConservation(env e)
    ghostNormalizedStables128
    == ghostCollateralNormalizedStables216[ghostCollateralList[0]]
    + ghostCollateralNormalizedStables216[ghostCollateralList[1]]
filtered { f -> !EXCLUDED_FUNCTION(f) } { preserved with (env eFunc) { SETUP_P(e, eFunc); } }
```

Result: 18/18 mutations caught — the invariant catches every injected bug across all three contracts.

Swapper._swap() mutations (8/8 caught)

```
certoraMutate certora/confs/Parallelizer/Swapper/valid_state_mutations_normalizedStablesSumConservation.conf
```



X [#1](#) Remove global NS update in mint.

```
// mint path
- ts.normalizedStables = ts.normalizedStables + changeAmount;
+ // removed
```

X [#2](#) Remove per-collateral NS update in mint.

```
// mint path
- collatInfo.normalizedStables = collatInfo.normalizedStables + uint216(changeAmount);
+ // removed
```

X [#3](#) Flip + to – for global NS in mint.

```
// mint path
- ts.normalizedStables = ts.normalizedStables + changeAmount;
+ ts.normalizedStables = ts.normalizedStables - changeAmount;
```

X [#4](#) Flip + to – for per-collateral NS in mint.

```
// mint path
- collatInfo.normalizedStables = collatInfo.normalizedStables + uint216(changeAmount);
+ collatInfo.normalizedStables = collatInfo.normalizedStables - uint216(changeAmount);
```

X [#5](#) Off-by-one for global NS in mint.

```
// mint path
- ts.normalizedStables = ts.normalizedStables + changeAmount;
+ ts.normalizedStables = ts.normalizedStables + changeAmount + 1;
```

X [#6](#) Remove global NS update in burn.

```
// burn path
- ts.normalizedStables = ts.normalizedStables - changeAmount;
+ // removed
```

✗ #7 Remove per-collateral NS update in burn.

```
// burn path
- collatInfo.normalizedStables = collatInfo.normalizedStables - uint216(changeAmount);
+ // removed
```

✗ #8 Flip - to + for global NS in burn.

```
// burn path
- ts.normalizedStables = ts.normalizedStables - changeAmount;
+ ts.normalizedStables = ts.normalizedStables + changeAmount;
```

certora Mutation testing

1 Rules

Q Search input text

1 Successful

R1 (8) — normalizedStablesSumCo...

0 Unsuccessful

100 %

Coverage

Coverage Map

Patch 8

certora/mutations/normalizedStablesSumConservation/Swapper/8.sol

Full report

```

10 > // from this collateral
11 > collatInfo.normalizedStables = collatInfo.normalizedStables - u
12 > - ts.normalizedStables = ts.normalizedStables - changeAmount;
13 > + ts.normalizedStables = ts.normalizedStables + changeAmount; //
14 > ITokenP(tokenIn).burnSelf(amountIn, msg.sender);
15 > if (collatInfo.isManaged > 0) {
16 >     LibManager.release(tokenOut, to, amountOut, collatInfo.managed
17 > ***** Mutation Diff Block End *****
18 >
19 238c255
20 < ts.normalizedStables = ts.normalizedStables - changeAmount;
21 ---
22 > ts.normalizedStables = ts.normalizedStables + changeAmount; // M

```

Show more

8 Mutants

Q Search input text

8 Caught

Patch 1

Patch 2

Patch 3

Patch 4

Patch 5

Patch 6

Patch 7

Patch 8

0 Uncaught

LibSetters.adjustStablecoins() mutations (6/6 caught)

```
certoraMutate certora/confs/Parallelizer/Setter/valid_state_mutations_normalizedStablesSumConservation.conf
```

✗ #1 Remove global NS update in increase.

```
if (increase) {
    collatInfo.normalizedStables = collatInfo.normalizedStables + uint216(normalizedAmount);
- ts.normalizedStables = ts.normalizedStables + normalizedAmount;
+ // removed
```

✗ #2 Remove per-collateral NS update in increase.

```

if (increase) {
-   collatInfo.normalizedStables = collatInfo.normalizedStables + uint216(normalizedAmount);
+   // removed
    ts.normalizedStables = ts.normalizedStables + normalizedAmount;

```

X #3 Flip + to – for global NS in increase.

```

if (increase) {
    collatInfo.normalizedStables = collatInfo.normalizedStables + uint216(normalizedAmount);
-   ts.normalizedStables = ts.normalizedStables + normalizedAmount;
+   ts.normalizedStables = ts.normalizedStables - normalizedAmount;

```

X #4 Remove global NS update in decrease.

```

} else {
    collatInfo.normalizedStables = collatInfo.normalizedStables - uint216(normalizedAmount);
-   ts.normalizedStables = ts.normalizedStables - normalizedAmount;
+   // removed

```

X #5 Remove per-collateral NS update in decrease.

```

} else {
-   collatInfo.normalizedStables = collatInfo.normalizedStables - uint216(normalizedAmount);
+   // removed
    ts.normalizedStables = ts.normalizedStables - normalizedAmount;

```

X #6 Flip – to + for global NS in decrease.

```

} else {
    collatInfo.normalizedStables = collatInfo.normalizedStables - uint216(normalizedAmount);
-   ts.normalizedStables = ts.normalizedStables - normalizedAmount;
+   ts.normalizedStables = ts.normalizedStables + normalizedAmount;

```

Redeemer._updateNormalizer() mutations (4/4 caught)

```
certoraMutate certora/confs/Parallelizer/Redeemer/valid_state_mutations_normalizedStablesSumConservation.conf
```

X #1 Remove global NS assignment after renormalization.

```

}
-   ts.normalizedStables = newNormalizedStables;
+   // removed

```

X #2 Remove per-collateral NS update in renormalization loop.

```

newNormalizedStables += newCollateralNormalizedStable;
-   ts.collaterals[collateralListMem[i]].normalizedStables = uint216(newCollateralNormalizedStable);
+   // removed

```

X #3 Remove accumulator update in renormalization loop.

```

-   newNormalizedStables += newCollateralNormalizedStable;
+   // removed
    ts.collaterals[collateralListMem[i]].normalizedStables = uint216(newCollateralNormalizedStable);

```

X #4 Double accumulator in renormalization.

```
- newNormalizedStables += newCollateralNormalizedStable;  
+ newNormalizedStables += 2 * newCollateralNormalizedStable;
```

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Common Setup (Steps 1–4)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup [tutorial](#).

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install [pipx](#) — installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match a specific prover version, pin it explicitly (e.g. `certora-cli==8.13.0`)

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.28
solc-select use 0.8.28
```

Remote Execution

5. Set up Certora key. You can get a free key through the Certora [Discord](#) or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

Note: If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:

```
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf --server production
```

Local Execution

Follow the full build instructions in the [CertoraProver repository \(v8.13.0\)](#). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
#   z3:   https://github.com/Z3Prover/z3/releases
#   cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm
```



```
# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.13.0
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Required source code modifications

The Certora Prover does not fully support arrays whose elements pack into a single storage slot (e.g. `uint64[]` where two elements share one 256-bit word). The Parallel codebase stores fee-curve breakpoints as `uint64[]` and `int64[]` arrays, which triggers incorrect INDEX hook behavior during verification. To work around this limitation, all packed fee arrays were widened to `uint256[] / int256[]`, and explicit `int64(...)` downcasts were added where narrower types are required at use sites. All changes are marked with `// CERTORA:` comments and are semantically neutral — values stay within their original 64-bit range.

1. **Storage.sol** — storage type widening (lines 119–122, 136–139)

- `Collateral.xFeeMint: uint64[] → uint256[]`
- `Collateral.yFeeMint: int64[] → int256[]`
- `Collateral.xFeeBurn: uint64[] → uint256[]`
- `Collateral.yFeeBurn: int64[] → int256[]`
- `ParallelizerStorage.xRedemptionCurve: uint64[] → uint256[]`
- `ParallelizerStorage.yRedemptionCurve: int64[] → int256[]`

2. **LibHelpers.sol** — function signature widening + downcasts (lines 55, 94–98)

- `findLowerBound param array: uint64[] → uint256[]`
- `piecewiseLinear params xArray/yArray: uint64[]/int64[] → uint256[]/int256[]`
- Three return-value downcasts added: `int64(yArray[...])` (lines 96, 97, 98)

3. **LibSetters.sol** — function signatures, events, local variables (lines 30, 34, 228, 244, 319, 358, 365)

- Events `FeesSet`, `RedemptionCurveParamsSet`: params `uint64[]/int64[] → uint256[]/int256[]`
- Functions `setFees`, `setRedemptionCurveParams`, `checkFees`: params `uint64[]/int64[] → uint256[]/int256[]`
- Local variables `burnFees`, `mintFees` inside `checkFees: int64[] → int256[]`

4. **Swapper.sol** — downcasts at fee lookup (lines 447, 595)

- Two `int64(...)` downcasts added on `collatInfo.yFeeMint[0] / collatInfo.yFeeBurn[0]` and `[n-1]` accesses

5. **Redeemer.sol** — local variable widening + downcast (lines 284, 288, 299)

- `yRedemptionCurveMem: int64[] → int256[]`

- `xRedemptionCurveMem`: `uint64[]` → `uint256[]`
- Downcast `int64(yRedemptionCurveMem[...])` at penalty-factor calculation

6. **Getters.sol** — return type widening (lines 52, 62, 72)

- `getCollateralMintFees` returns: `uint64[] / int64[]` → `uint256[] / int256[]`
- `getCollateralBurnFees` returns: `uint64[] / int64[]` → `uint256[] / int256[]`
- `getRedemptionFees` returns: `uint64[] / int64[]` → `uint256[] / int256[]`

7. **IGetters.sol** — interface return type widening (lines 33, 37, 41)

- Same three getters as above

8. **ISetters.sol** — interface parameter widening (lines 90, 93)

- `setFees`, `setRedemptionCurveParams`: `params uint64[] / int64[]` → `uint256[] / int256[]`

9. **SettersGuardian.sol** — parameter widening (lines 28, 33)

- `setFees`, `setRedemptionCurveParams`: `params uint64[] / int64[]` → `uint256[] / int256[]`

Running Verification

Build the project first:

```
cd Parallel-Tokens && bun install --frozen-lockfile && forge build && cd ..
cd parallel-parallelizer && bun install --frozen-lockfile && forge build && cd ..
```

Quick Runs

`variable_transitions` and `unit_tests` are lightweight and can be run as full conf files:

```
certoraRun certora/confs/FlashParallelToken/variable_transitions.conf
certoraRun certora/confs/FlashParallelToken/unit_tests.conf
certoraRun certora/confs/Parallelizer/Swapper/variable_transitions.conf
certoraRun certora/confs/Parallelizer/Swapper/unit_tests.conf
certoraRun certora/confs/Parallelizer/Redeemer/variable_transitions.conf
certoraRun certora/confs/Parallelizer/Redeemer/unit_tests.conf
certoraRun certora/confs/Parallelizer/Setter/variable_transitions.conf
certoraRun certora/confs/Parallelizer/Setter/unit_tests.conf
certoraRun certora/confs/Parallelizer/Surplus/variable_transitions.conf
certoraRun certora/confs/Parallelizer/Surplus/unit_tests.conf
certoraRun certora/confs/Savings/variable_transitions.conf
certoraRun certora/confs/Savings/unit_tests.conf
certoraRun certora/confs/Savings/erc4626.conf
```

Note: `valid_state`, `state_transitions`, and `high_level` may time out when running all rules at once. Run them per-rule using `--rule` as shown below.

FlashParallelToken

```
# Valid State (1 invariant)
certoraRun certora/confs/FlashParallelToken/valid_state.conf \
  --rule feeRateNeverExceedsMax

# State Transitions (4 rules)
certoraRun certora/confs/FlashParallelToken/state_transitions.conf \
  --rule tokensOnlyWithdrawnToFeeRecipient
certoraRun certora/confs/FlashParallelToken/state_transitions.conf \
  --rule feeRecipientAndTokenSettingsChangeSeparately
certoraRun certora/confs/FlashParallelToken/state_transitions.conf \
  --rule tokenSettingsDoNotAffectOtherTokens
certoraRun certora/confs/FlashParallelToken/state_transitions.conf \
  --rule configChangesDoNotMoveTokens
```

```
# High-Level (4 rules)
certoraRun certora/confs/FlashParallelToken/high_level.conf \
  --rule flashLoanDoesNotChangeTotalSupply
certoraRun certora/confs/FlashParallelToken/high_level.conf \
  --rule flashLoanCollectsExactFee
certoraRun certora/confs/FlashParallelToken/high_level.conf \
  --rule borrowerCannotProfitFromFlashLoan
certoraRun certora/confs/FlashParallelToken/high_level.conf \
  --rule flashLoanFeeNeverExceedsBorrowedAmount
```

Parallelizer — Valid State (32 invariants, verified across all 4 facets)

```
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralListNoZeroValues
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralListNoDuplicates
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralListDecimalsNonZero
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule normalizerBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule slippageToleranceBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule surplusBufferRatioBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule lastReleasedAtBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule isRedemptionLiveFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule isTrustedFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule isSellerTrustedFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralIsManagedFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralIsMintLiveFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralIsBurnLiveFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralOnlyWhitelistedFlag
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule normalizedStablesSumConservation
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule feeMintLengthMatching
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule feeBurnLengthMatching
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule redemptionCurveLengthMatching
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule feeMintXOrdering
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule feeBurnXOrdering
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule totalSharesSumConservation
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule payeesNoDuplicates
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule payeeSharesPositive
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralDecimalsMatchErc20
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
  --rule collateralDataZeroOutsideList
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
```

```

--rule sharesZeroOutsidePayees
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
--rule feeMintYBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
--rule feeBurnYBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
--rule redemptionCurveYBounded
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
--rule feeMintXAnchor
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
--rule feeBurnXAnchor
certoraRun certora/confs/Parallelizer/Swapper/valid_state.conf \
--rule redemptionCurveXBounded

```

Note: The same 32 invariants are verified across all four facets. Replace `Swapper` with `Redeemer`, `Setter`, or `Surplus` to run against other facets.

Parallelizer — State Transitions (13 rules, verified across all 4 facets)

```

certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule renormalizationResetsNormalizerToBase27
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule perCollateralNsChangeImpliesGlobalNsChange
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule globalNsChangeImpliesSomeCollatNsChange
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule multipleCollateralNsChangesImplyRenormalization
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule collateralDecimalsAndListAtomicity
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule collateralListGrowthPreservesTokenPSupply
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule tokenPSupplyChangeImpliesProtocolStateChange
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule stablecoinCapEnforcedOnNsIncrease
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule tokenPSupplyIncreaseImpliesNsIncrease
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule diamondTokenPBalanceChangeImpliesSurplusOrRecoveryPath
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule accountingChangePreservesConfiguration
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule lastReleasedAtOnlyUpdatesToTimestamp
certoraRun certora/confs/Parallelizer/Swapper/state_transitions.conf \
--rule collateralListShrinkImpliesZeroNs

```

Note: Replace `Swapper` with `Redeemer`, `Setter`, or `Surplus` to run against other facets.

Parallelizer — Swapper High-Level (19 rules)

```

certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule mintExactInputSupplyDelta
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule burnExactInputSupplyDelta
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule mintExactOutputSupplyDelta
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule burnExactOutputSupplyDelta
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule mintCollateralTransfer
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule burnCollateralTransfer
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \

```

```

--rule swapOnlyAffectsTargetCollateral
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule zeroAmountInNoOp
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule mintExactInputIncreasesNs
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule burnExactInputDecreasesNs
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule mintExactOutputIncreasesNs
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule burnExactOutputDecreasesNs
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule swapExactInputSlippageGuarantee
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule swapExactOutputSlippageGuarantee
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule zeroAmountOutExactOutputNoOp
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule quoteInMatchesMintExactInput
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule quoteInMatchesBurnExactInput
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule quoteInSameForAnyCaller
certoraRun certora/confs/Parallelizer/Swapper/high_level.conf \
--rule quoteOutSameForAnyCaller

```

Parallelizer — Redeemer High-Level (10 rules)

```

certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule updateNormalizerIncreaseGrowsStablecoinsIssued
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule updateNormalizerDecreaseShrinksStablecoinsIssued
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule updateNormalizerIncreaseRaisesNormalizer
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule updateNormalizerDecreaseLowersNormalizer
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule redeemDecreasesStablecoinsIssued
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule largerRedeemDecreasesStablecoinsIssuedMore
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule largerUpdateNormalizerIncreasesMore
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule updateNormalizerIncreaseBoundedByAmount
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule updateNormalizerDecreaseBoundedByAmount
certoraRun certora/confs/Parallelizer/Redeemer/high_level.conf \
--rule redeemWithoutRenormPreservesNormalizedStables

```

Parallelizer — Setter High-Level (9 rules)

```

certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule pauseUnpauseMintRoundTrip
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule pauseUnpauseBurnRoundTrip
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule pauseUnpauseRedeemRoundTrip
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule toggleTrustedUpdaterRoundTrip
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule toggleTrustedSellerRoundTrip
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule adjustStablecoinsRoundTrip

```

```
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule adjustStablecoinsIncreaseMonotonicity
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule adjustStablecoinsDecreaseMonotonicity
certoraRun certora/confs/Parallelizer/Setter/high_level.conf \
--rule adjustStablecoinsGlobalLocalDeltaConservation
```

Parallelizer — Surplus High-Level (10 rules)

```
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusIncreasesTokenPSupply
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusCreditsTokenPToDiamond
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule releaseDistributesAllTokenP
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusIncreasesNormalizedStables
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusIncreasesPerCollateralNs
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusPositiveIssuedImpliesNsIncrease
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule surplusLifecycleProcessThenRelease
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusCapsToMaxCollateral
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusReachableWithHighBufferRatio
certoraRun certora/confs/Parallelizer/Surplus/high_level.conf \
--rule processSurplusManagedCollateralIsReachable
```

Savings

```
# Valid State (4 invariants)
certoraRun certora/confs/Savings/valid_state.conf \
--rule pausedFlagIsZeroOrOne
certoraRun certora/confs/Savings/valid_state.conf \
--rule trustedUpdaterFlagIsZeroOrOne
certoraRun certora/confs/Savings/valid_state.conf \
--rule lastUpdateNeverExceedsBlockTime
certoraRun certora/confs/Savings/valid_state.conf \
--rule vaultNeverApprovesOwnShares

# State Transitions (3 rules)
certoraRun certora/confs/Savings/state_transitions.conf \
--rule stateChangeAlwaysRefreshesTimestamp
certoraRun certora/confs/Savings/state_transitions.conf \
--rule pausedContractCannotChangeShareSupply
certoraRun certora/confs/Savings/state_transitions.conf \
--rule rateCannotIncreaseAboveMaxRate

# High-Level (17 rules)
certoraRun certora/confs/Savings/high_level.conf \
--rule depositThenRedeemCannotProfit
certoraRun certora/confs/Savings/high_level.conf \
--rule mintThenRedeemCannotProfit
certoraRun certora/confs/Savings/high_level.conf \
--rule depositThenWithdrawCannotProfit
certoraRun certora/confs/Savings/high_level.conf \
--rule withdrawThenDepositCannotProfit
certoraRun certora/confs/Savings/high_level.conf \
--rule redeemThenMintCannotProfit
certoraRun certora/confs/Savings/high_level.conf \
--rule mintThenWithdrawCannotProfit
```

```
certoraRun certora/confs/Savings/high_level.conf \
--rule moreAssetsDepositedMeansMoreShares
certoraRun certora/confs/Savings/high_level.conf \
--rule moreSharesRedeemedMeansMoreAssets
certoraRun certora/confs/Savings/high_level.conf \
--rule moreSharesMintedCostsMoreAssets
certoraRun certora/confs/Savings/high_level.conf \
--rule moreAssetsWithdrawnBurnsMoreShares
certoraRun certora/confs/Savings/high_level.conf \
--rule depositDoesNotDiluteShares
certoraRun certora/confs/Savings/high_level.conf \
--rule mintDoesNotDiluteShares
certoraRun certora/confs/Savings/high_level.conf \
--rule redeemDoesNotDiluteShares
certoraRun certora/confs/Savings/high_level.conf \
--rule withdrawDoesNotDiluteShares
certoraRun certora/confs/Savings/high_level.conf \
--rule depositIsAdditive
certoraRun certora/confs/Savings/high_level.conf \
--rule donationDoesNotInflateShareValue
certoraRun certora/confs/Savings/high_level.conf \
--rule donationDoesNotAmplifyTotalAssets

# ERC4626 donation-resistance — extended solver portfolio for the
# proportional-capture bound (proposed EIP-4626-51 addition).
# Standalone conf with smt_timeout=14400 and parallel multi-solver search.
certoraRun certora/confs/Savings/erc4626_donationProfitBoundedByShareRatio.conf
```

Resources

To learn more about Certora formal verification:

- [Updraft Assembly & Formal Verification Course](#) -- Comprehensive video course covering assembly and formal verification from the ground up
- [Find Highs Using Certora Formal Verification](#) -- Practical guide with a companion [repo](#) containing simplified examples based on real code and bugs from private audits
- [RareSkills Certora Book](#) -- Structured tutorial covering CVL syntax, patterns, and common pitfalls
- [AlexZoid FV Resources](#) -- Curated collection of formal verification resources, examples, and references
- [Certora Tutorials](#) -- Official Certora documentation and guided tutorials